

DineDash Food Delivery Data Analysis

by Alankrit Agarwal

Problem Statements and Requirements:

1. Background

DineDash food delivery industry has seen rapid growth over the last decade, fueled by the increasing demand for convenience, improved delivery logistics, and the expansion of digital platforms. MealDash have revolutionized how food is ordered and delivered, creating new opportunities for restaurant owners and consumers alike.

However, as the industry continues to expand, food delivery platforms face increasing competition, customer expectations for faster and more personalized service, and challenges in optimizing their operations across a diverse and extensive menu offering.

In this case study, we focus on the use of **big data and advanced analytics** to streamline operations, optimize delivery times, enhance customer experience, and improve menu offerings. By leveraging data processing platforms such as **Apache Spark** and **Databricks**, we can better understand customer preferences, predict demand, and offer personalized recommendations, ultimately driving more efficient and profitable business operations.

2. Challenges

Dinedash has to address several challenges to maintain competitive advantage and operational efficiency:

- **Data Fragmentation:** Different systems for orders, customer profiles, menus, and delivery logs lead to fragmented data that is difficult to analyze comprehensively.
- **Customer Segmentation:** Understanding customer preferences based on historical data to offer personalized promotions, menu recommendations, or loyalty rewards.
- **Order Prediction:** Predicting order volume in different regions at different times of the day to optimize delivery capacity and reduce wait times.
- **Menu Optimization:** Identifying the most popular and profitable menu items while suggesting item combinations to customers. Ensuring the menu is always updated in real-time for availability.
- **Order Fulfillment Delays:** Ensuring on-time deliveries and improving operational workflows, which are often impacted by external factors like weather, traffic, or restaurant delays.
- **Promotions and Discounts:** Identifying the most effective promotional offers and discounts that increase order value and drive repeat purchases without eroding profitability.

3. Business Needs

To stay competitive and meet customer demands, DineDash needs to address the following:

- **Data Integration:** Integrating data from different sources (customer orders, restaurant data, delivery times, etc.) to build a comprehensive view of customer behavior, restaurant performance, and delivery operations.
- **Real-Time Analytics:** Implementing a real-time data processing pipeline that analyzes orders as they come in, providing instant insights and enabling dynamic decision-making (e.g., dynamic pricing, offer recommendations).
- **Predictive Analytics:** Using machine learning to predict demand, customer preferences, delivery time, and possible bottlenecks in the system.
- **Optimization Algorithms:** Implementing algorithms that optimize the delivery process by calculating the best route, available delivery agents, and reducing food delivery time.
- **Personalization:** Offering personalized promotions, product recommendations, and even tailored menus based on past customer behavior, location, and preferences.
- **Performance Tracking:** Continuously monitoring key performance indicators (KPIs), such as customer satisfaction, order fulfillment rate, delivery time, and revenue per order.

4. Proposed Solution

The proposed solution involves setting up a big data processing pipeline using Databricks and Apache Spark, along with machine learning models to provide real-time insights and predictive analytics. The solution leverages:

- **Data Lake:** A centralized repository for storing all structured and unstructured data, including order details, customer information, menu data, and delivery logs.
- **ETL Pipelines:** Using Delta Live Tables (DLT) to manage the transformation of raw data into usable formats for analysis. The pipeline will clean, enrich, and structure the data for downstream analytics.
- **Real-Time Streaming:** Processing incoming orders in real-time using structured streaming to calculate metrics like order volume, delivery times, and detect any operational anomalies as they occur.
- **Analytics Dashboards:** Developing dashboards to visualize customer trends, restaurant performance, order patterns, and delivery efficiency.
- **Machine Learning Models:** Implementing models for demand forecasting, customer segmentation, and personalized recommendations to boost sales and improve customer experience.
- **Optimized Delivery Routing:** Using geographic data and algorithms to optimize delivery routes and reduce delivery times.

We will focus upon ETL Pipelines, Streaming Data analytics and Dashboards.

5. Attributes

DineDash will track the following key attributes across different tables:

1. Customers

- **customer_id:** Unique identifier for each customer.
- **name:** Name of the customer.
- **email:** Email address of the customer.
- **signup_date:** Date customer signed up

2. Restaurants

- **restaurant_id:** Unique identifier for each restaurant.
- **name:** Name of the restaurant.
- **cuisine:** Type of cuisine (e.g., Italian, Chinese, Indian).
- **location_id:** Geographic location (latitude/longitude) of the restaurant

3. Menu Items

- **item_id:** Unique identifier for each menu item.
- **restaurant_id:** Foreign key referencing restaurants.
- **item_name:** Name of the menu item (e.g., "Margherita Pizza").
- **price:** Price of the menu item.

4. Orders

- **order_id:** Unique identifier for each order.
- **timestamp:** Time the order was placed.
- **customer_id:** Foreign key referencing customers.
- **restaurant_id:** Foreign key referencing restaurants.
- **agent_id:** [Identifier for the delivery agent]
- **delivery_location_id:** [Identifier for the delivery location]
- **items_ordered:** List of items in the order (can be a list of menu_item_ids).
- **Total_amount:** [Total cost of the order]
- **Tip:** [Gratuity amount]
- **payment_method:** Payment method used (e.g., "Credit Card", "Cash").
- **order_status:** Status of the order (e.g., "Pending", "Delivered").

5. Delivery Agent

- **agent_id:** Unique identifier for each delivery [agent].

- **name:** [Name of the delivery agent]
- **phone:** [Contact number]
- **rating:** [Agent's rating]

Notebooks, Schema, Volumes, Dashboard, Pipelines, Workflow made:

Schema:

1. **alan_dinedash_db:** Used as the main schema for the volume in which the dataset is being uploaded
2. **alan_dinedash_dlt_db:** Used while running of ETL pipeline

Volume:

- **alan_dinedash_raw:** Used under alan_dinedash_db for the dataset upload
(*Session_Id e.g: 7405606622859005 sometimes change due to New Workspace*)
 - **Volume Root Path:**
/Volumes/alankrit_db_workspace_7405606622859005/alan_dinedash_db
/alan_dinedash_raw (dimensions and orders directory here)
 - **Orders File:**
/Volumes/alankrit_db_workspace_7405606622859005/alan_dinedash_db
/alan_dinedash_raw/orders (stream_data; i.e; Orders data inside it)

Notebooks:

1. **Temporary Views Notebook:** Used for creation of temporary views of the attributes of the DineDash.
2. **alan_DineDash_DLT_Pipeline:**
 - a. Used for the writing of Bronze Tables of the attributes of the DineDash.
 - b. Used for the writing of Filtered Tables of the attributes of the DineDash.
 - c. Used for the creation of alan_orders_bronze table and execution of COPY INTO command in it.
3. **alan_Dinedash_SQL_Analysis:** Used for the execution of Orders Data Analysis and Orders & Dimensions Data Analysis.
4. **alan_DineDash_DLT_PL:** Used as ETL Pipeline execution notebook.

Pipeline:

- **alan_dinedash_etl_pipeline:** Used for execution of alan_DineDash_DLT_PL notebook for demonstration of ETL pipeline.

SQL Queries:

1. **Total Orders per Restaurant:** Query Output and Visualization (Bar Chart)
2. **Top Selling Items:** Query Output and Visualization (Pie Chart)
3. **Daily Revenue Trend:** Query Output and Visualization (Line Chart)
4. **Total Revenue:** Query Output and Visualization (Counter with '\$' format)

Dashboard:

- **DineDash Metrics Dashboard:** Used for the showcase of visualization of orders data of the months in the form of Bar, Pie and Line charts etc.

Workflow:

- **DineDash Workflow:** Used for automation of ETL pipeline connection with Dashboard directly to feed the data incrementally and observe of change in data in Dashboard automatically.

Data Ingestion via Unity Catalog Volumes:

Data Ingestion

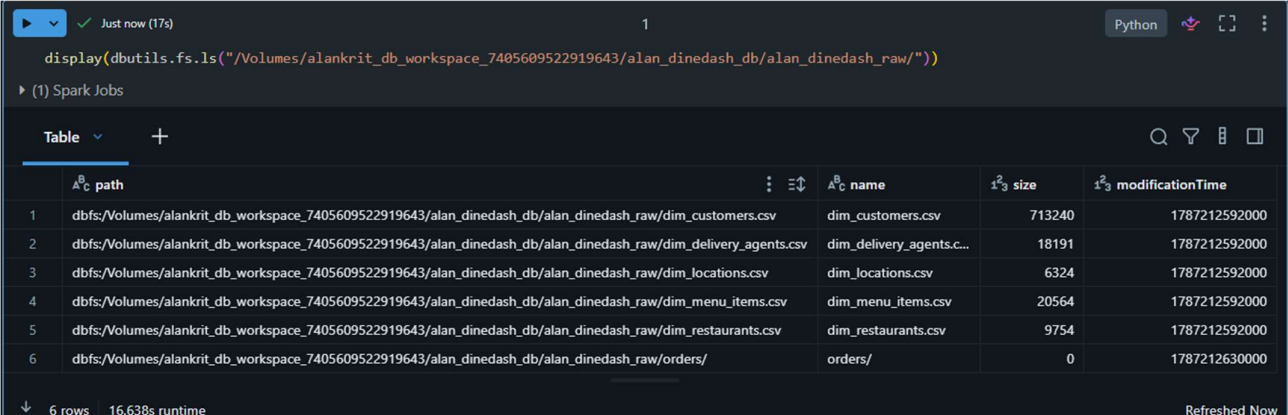
Manually Uploading datasets to DBFS

In **enb-accenture** databricks workspace

Create folder in DBFS as `/FileStore/tables/<enterpriseid>/dinedash` and upload the dataset as shown below.

For orders create a folder by the name **orders** within **dinedash** folder and upload only Jan and Feb orders datasets to begin with

Solution: Created a Unity-Catalog Volume to replace legacy DBFS storage, adhering to enterprise security best practices. Uploaded 5-dimensional CSV files to the root volume and staged the first two months of transactional order data (JSON) into an orders subdirectory.



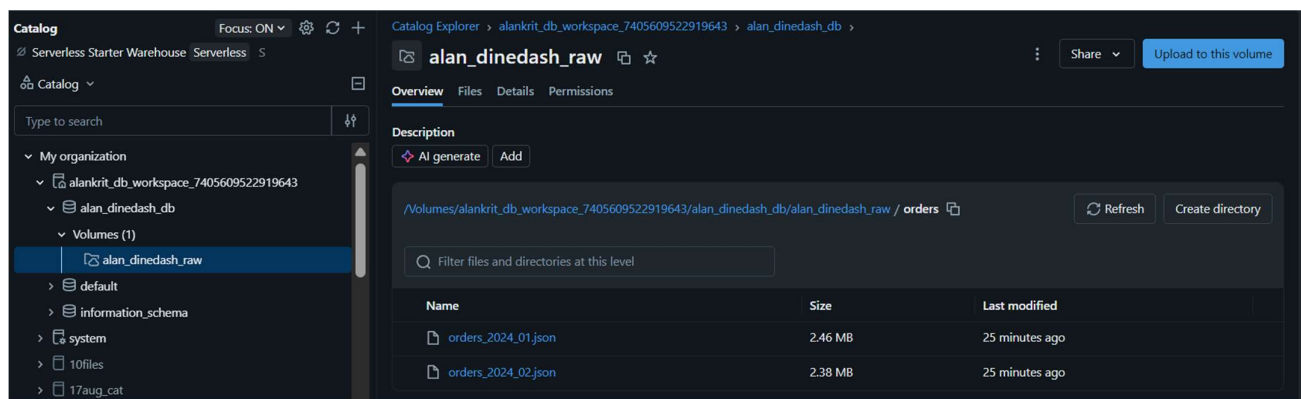
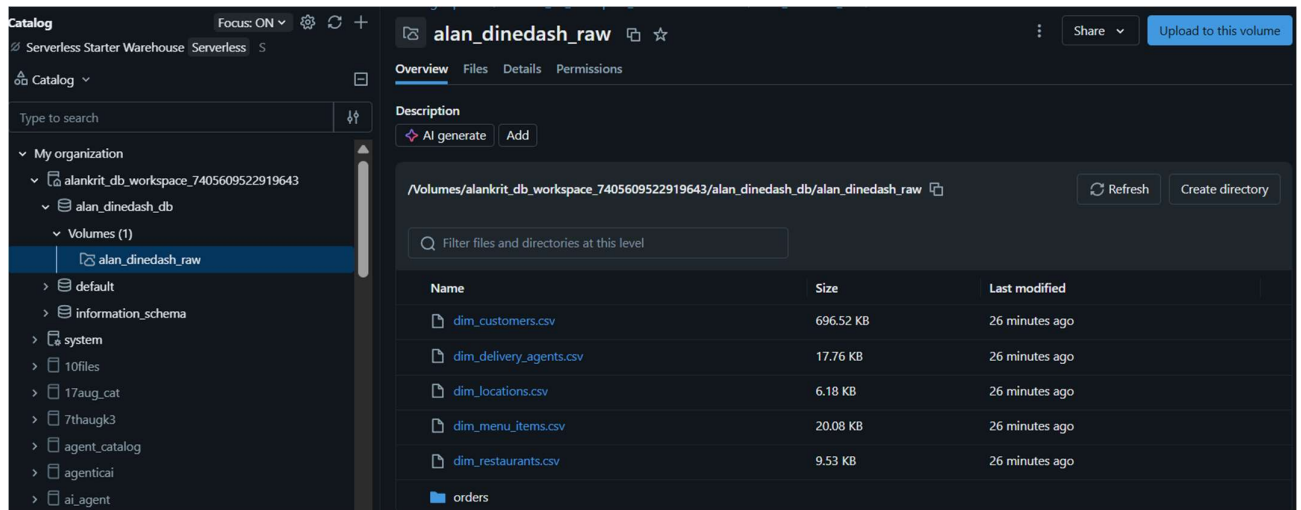
```
display(dbutils.fs.ls("/Volumes/alkankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/"))
```

(1) Spark Jobs

	path	name	size	modificationTime
1	dbfs:/Volumes/alkankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_customers.csv	dim_customers.csv	713240	1787212592000
2	dbfs:/Volumes/alkankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_delivery_agents.csv	dim_delivery_agents.c...	18191	1787212592000
3	dbfs:/Volumes/alkankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_locations.csv	dim_locations.csv	6324	1787212592000
4	dbfs:/Volumes/alkankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_menu_items.csv	dim_menu_items.csv	20564	1787212592000
5	dbfs:/Volumes/alkankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_restaurants.csv	dim_restaurants.csv	9754	1787212592000
6	dbfs:/Volumes/alkankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/orders/	orders/	0	1787212630000

6 rows 16.638s runtime Refreshed Now

Screenshot: dbutils.fs.ls() query showing the uploaded files in `alan_dinedash_raw` volume.



Screenshots: Azure Databricks Catalog Explorer showing the alan_dinedash_raw volume and the uploaded files.

Problem Statement:

Initial Data Exploration

Querying the files directly

Explore each of the dataset by querying against the DBFS path of each dataset files directly. Observe the sample records to understand about the dataset by displaying all the columns for 10 records.

Solution: We will use Unity-Catalog Volume to replace legacy DBFS storage.

1. Creating Temporary Views:

1. Create temporary views using the `read_files()` table-valued function by inferring schema from the data for each of the datasets for customers, restaurants, locations, delivery_agents, menu_items as

```
customers_tmp_view,  
restaurants_tmp_view,  
locations_tmp_view,  
delivery_agents_tmp_view,  
menu_items_tmp_view
```

Solution: Created temporary SQL views on top of the raw CSV files stored in the Unity Catalog Volume. By utilizing the `read_files` function with `inferSchema => true` and `header => true`, I queried the unstructured data directly to ensure columns and data types aligned with expected business formats.

****Attached all the screenshots for the same below****

- Creation of Customer Temp View using `dim_customers.csv`

The screenshot shows a SQL IDE interface. At the top, there's a status bar indicating '22 hours ago (27s)' and a tab labeled '1'. Below this, the SQL editor contains the following code:

```
CREATE OR REPLACE TEMPORARY VIEW alan_customers_tmp_view AS  
SELECT * FROM read_files(  
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_customers.csv',  
  format => 'csv',  
  header => true,  
  inferSchema => true  
);  
  
SELECT * FROM alan_customers_tmp_view LIMIT 10;
```

Below the SQL editor, there's a section for Spark Jobs, showing '(2) Spark Jobs'. Underneath, there's a table view of the data. The table has 8 columns: `customer_id`, `name`, `email`, `dob`, `signup_date`, `location_id`, and `rescued_data`. The data is displayed in a table with 10 rows, numbered 1 to 10.

	<code>customer_id</code>	<code>name</code>	<code>email</code>	<code>dob</code>	<code>signup_date</code>	<code>location_id</code>	<code>rescued_data</code>
1	C0001	Matthew Rodriguez...	heatherdouglass@example.org	1954-12-02	2019-05-16	L069	null
2	C0002	Justin Diaz	millernathan@example.net	1985-07-05	2018-09-06	L016	null
3	C0003	Patricia Pena	kristinesanchez@example.co...	2003-10-19	2012-05-21	L049	null
4	C0004	David Smith	cassandra07@example.org	1993-02-22	2003-03-29	L011	null
5	C0005	Meghan George	johnsonrandall@example.com	1956-05-17	2016-01-27	L071	null
6	C0006	Jennifer Beard	djohnson@example.net	1974-02-12	1991-01-17	L038	null
7	C0007	Tiffany Davis	thomasjoseph@example.com	1978-05-20	1998-04-16	L081	null
8	C0008	Rachel Elliott	pbarnes@example.com	1964-07-20	1997-04-21	L080	null
9	C0009	Mark Martin	ijohnson@example.org	1964-05-01	1980-04-16	L047	null
10	C0010	Julie Murphy	amy49@example.net	1968-05-28	2014-11-27	L074	null

- Creation of Restaurant Temp View using `dim_restaurants.csv`

22 hours ago (3s) 2

```

CREATE OR REPLACE TEMPORARY VIEW alan_restaurants_tmp_view AS
SELECT * FROM read_files(
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_restaurants.csv',
  format => 'csv',
  header => true,
  inferSchema => true
);

SELECT * FROM alan_restaurants_tmp_view LIMIT 10;

```

(2) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: string, name: string ... 5 more fields]

	restaurant_id	name	cuisines	location_id	rating	delivery_fee	_rescued_data
1	R001	Food Haven Grill	Indian	L069	3.3	3.9	null
2	R002	Spice Villa Hub	Thai,Italian,Chinese	L012	3.8	2.25	null
3	R003	null	Indian,Thai	L076	0	0	null
4	R004	Curry Leaf Hub	Indian,American	L048	3.1	3.03	null
5	R005	Tasty Bites Grill	Mexican	L039	3.9	1.02	null
6	R006	null	Thai,American	L077	0	0	null
7	R007	Pasta Palace Corner	American,Mexican,Chine...	L014	3.3	3.33	null
8	R008	Curry Leaf Grill	Chinese,Mexican	L053	2.7	1.25	null
9	R009	Curry Leaf Corner	Mexican	L010	3.4	3.11	null
10	R010	Pasta Palace Hub	Mexican,Italian,American	L082	3.6	2.76	null

- Creation of Locations Temp View using dim_locations.csv

22 hours ago (3s) 3

```

CREATE OR REPLACE TEMPORARY VIEW alan_locations_tmp_view AS
SELECT * FROM read_files(
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_locations.csv',
  format => 'csv',
  header => true,
  inferSchema => true
);

SELECT * FROM alan_locations_tmp_view LIMIT 10;

```

(2) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [location_id: string, area: string ... 5 more fields]

	location_id	area	city	state	latitude	longitude	_rescued_data
1	L001	Kirk Valley	North Sarah	Michigan	-2.8767365	-62.888285	null
2	L002	Cory Vista	New Edgar	Washington	-54.1725655	-46.757126	null
3	L003	Daryl Extensions	Port Justinmouth	Michigan	-19.072773	27.938702	null
4	L004	Grant Glen	Sellerston	Texas	13.0952805	141.629215	null
5	L005	Garcia Neck	Port Allison	Wisconsin	8.239121	26.958565	null
6	L006	Smith Rapids	Bobbyport	Missouri	16.931649	-49.121678	null
7	L007	Michael Streets	Duranport	West Virginia	-66.185224	-142.948244	null
8	L008	John Plains	Lake Kristinton	Oklahoma	-83.706852	28.380354	null
9	L009	Dennis Branch	North Brenda	Maryland	15.1168605	-67.825285	null
10	L010	Flores Vista	Port Joseph	Louisiana	43.557465	35.970725	null

- Creation of Delivery Agents Temp View using dim_delivery_agents.csv

22 hours ago (3s) 4

```
CREATE OR REPLACE TEMPORARY VIEW alan_delivery_agents_tmp_view AS
SELECT * FROM read_files(
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_delivery_agents.csv',
  format => 'csv',
  header => true,
  inferSchema => true
);

SELECT * FROM alan_delivery_agents_tmp_view LIMIT 10;
```

(2) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, name: string ... 3 more fields]

Table +

	A _C agent_id	A _C name	A _C phone	1.2 rating	A _C _rescued_data
1	A001	Lindsey Lang	649.286.1390	3.67	null
2	A002	Heidi Rodriguez	597.686.1508x2345	3.87	null
3	A003	Timothy Dudley	306.628.8953x804	4.57	null
4	A004	Victor Caldwell	null	0	null
5	A005	Rebecca Mullen	998-623-6945x94755	4.47	null
6	A006	William Sullivan	(430)391-1726	3.7	null
7	A007	Courtney Stevens	+1-715-785-0893	4.39	null
8	A008	Amy Patterson	(294)393-3957	4.12	null
9	A009	John Pham	001-319-364-9142x33514	4.91	null
10	A010	Sarah Melendez	444.257.2588	4.18	null

- Creation of Menu Items Temp View using dim_menu_items.csv

22 hours ago (3s) 5

```
CREATE OR REPLACE TEMPORARY VIEW alan_menu_items_tmp_view AS
SELECT * FROM read_files(
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_menu_items.csv',
  format => 'csv',
  header => true,
  inferSchema => true
);

SELECT * FROM alan_menu_items_tmp_view LIMIT 10;
```

(2) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [item_id: string, restaurant_id: string ... 4 more fields]

Table +

	A _C item_id	A _C restaurant_id	A _C item_name	1.2 price	A _C category	A _C _rescued_data
1	I0001	R017	Radio Soda	17.97	Beverage	null
2	I0002	R169	Occur Pizza	27.44	Main Course	null
3	I0003	R178	Road Nachos	9.5	Appetizer	null
4	I0004	R032	Detail Juice	5.98	Beverage	null
5	I0005	R042	Floor Nachos	7.2	Appetizer	null
6	I0006	R022	Benefit Ice Cream	29.5	Dessert	null
7	I0007	R140	Firm Cake	9.29	Dessert	null
8	I0008	R077	East Pizza	6.15	Main Course	null
9	I0009	R137	Same Brownie	16.12	Dessert	null
10	I0010	R134	Wrong Juice	0	Beverage	null

Screenshots: Temporary views creation and showing the successful output using SELECT * statements.

2. Creating Raw Bronze Delta Tables:

2. Create raw delta tables providing the manual schema and capture the record ingestion time and source file name for each record.

Create tables for each dataset, namely

customers_bronze,
restaurants_bronze,
locations_bronze,
delivery_agents_bronze
menu_items_bronze from

Ensure that all tables have appropriate column names and data types

Solution: Utilized CREATE OR REPLACE TABLE ... AS SELECT (CTAS) statements to seamlessly read from the temporary views and write to permanent Delta tables. During the projection, I explicitly casted data types (e.g., converting strings to standard DATE formats) to fulfil the manual schema requirement. I also generated ingestion_time and source_file_name metadata columns.

****Attached all the screenshots for the same below****

- Creation of Customers Bronze Table and its output

```
CREATE OR REPLACE TABLE alan_customers_bronze AS
SELECT
  CAST(customer_id AS STRING) AS customer_id,
  CAST(name AS STRING) AS name,
  CAST(email AS STRING) AS email,
  to_date(dob, 'yyyy-MM-dd') AS dob,
  to_date(signup_date, 'yyyy-MM-dd') AS signup_date,
  CAST(location_id AS STRING) AS location_id,
  current_timestamp() AS ingestion_time,
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_customers.
  csv' AS source_file_name
FROM read_files(
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_customers.
  csv',
  format => 'csv',
  header => true,
  inferSchema => true
);
```

12:42 PM (11s) 3 SQL

```
SELECT * FROM alan_customers_bronze;
```

(1) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, name: string ... 6 more fields]

	customer_id	name	email	dob	signup_date	location_id	ingestion_time	source_file_name
1	C0001	Matthew Rodriguez	heatherdouglass@example.org	1954-12-02	2019-05-16	L069	2026-08-20T09:57:35.645+00...	/Volumes/alankrit_c
2	C0002	Justin Diaz	millernathan@example.net	1985-07-05	2018-09-06	L016	2026-08-20T09:57:35.645+00...	/Volumes/alankrit_c
3	C0003	Patricia Pena	kristinesanchez@example.com	2003-10-19	2012-05-21	L049	2026-08-20T09:57:35.645+00...	/Volumes/alankrit_c
4	C0004	David Smith	cassandra07@example.org	1993-02-22	2003-03-29	L011	2026-08-20T09:57:35.645+00...	/Volumes/alankrit_c
5	C0005	Meghan George	johnsonrandall@example.com	1956-05-17	2016-01-27	L071	2026-08-20T09:57:35.645+00...	/Volumes/alankrit_c
6	C0006	Jennifer Beard	djohnson@example.net	1974-02-12	1991-01-17	L038	2026-08-20T09:57:35.645+00...	/Volumes/alankrit_c
7	C0007	Tiffany Davis	thomasjoseph@example.com	1978-05-20	1998-04-16	L081	2026-08-20T09:57:35.645+00...	/Volumes/alankrit_c
8	C0008	Rachel Elliott	pbarnes@example.com	1964-07-20	1997-04-21	L080	2026-08-20T09:57:35.645+00...	/Volumes/alankrit_c
9	C0009	Mark Martin	ijohnson@example.org	1964-05-01	1980-04-16	L047	2026-08-20T09:57:35.645+00...	/Volumes/alankrit_c
10	C0010	Julie Murphy	amy49@example.net	1968-05-28	2014-11-27	L074	2026-08-20T09:57:35.645+00...	/Volumes/alankrit_c

- Creation of Restaurant Bronze Table and it's output

03:29 PM (4s) 3

```
CREATE OR REPLACE TABLE alan_restaurants_bronze AS
SELECT
  CAST(restaurant_id AS STRING) AS restaurant_id,
  CAST(name AS STRING) AS name,
  CAST(cuisines AS STRING) AS cuisines,
  CAST(location_id AS STRING) AS location_id,
  CAST(rating AS DOUBLE) AS rating,
  CAST(delivery_fee AS DOUBLE) AS delivery_fee,
  current_timestamp() AS ingestion_time,
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_restaurants.
  csv' AS source_file_name
FROM read_files(
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_restaurants.
  csv',
  format => 'csv',
  header => true,
  inferSchema => true
);
```

(3) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long]

Just now (6s) 5

SQL

```
SELECT * FROM alan_restaurants_bronze;
```

(1) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: string, name: string ... 6 more fields]

	restaurant_id	name	cuisines	location_id	rating	delivery_fee	ingestion_time	source_file_name
1	R001	Food Haven Grill	Indian	L069	3.3	3.9	2026-08-20T09:59:13.445+00:...	/Volumes/alankrit_db_wo
2	R002	Spice Villa Hub	Thai,Italian,Chinese	L012	3.8	2.25	2026-08-20T09:59:13.445+00:...	/Volumes/alankrit_db_wo
3	R003	null	Indian,Thai	L076	0	0	2026-08-20T09:59:13.445+00:...	/Volumes/alankrit_db_wo
4	R004	Curry Leaf Hub	Indian,American	L048	3.1	3.03	2026-08-20T09:59:13.445+00:...	/Volumes/alankrit_db_wo
5	R005	Tasty Bites Grill	Mexican	L039	3.9	1.02	2026-08-20T09:59:13.445+00:...	/Volumes/alankrit_db_wo
6	R006	null	Thai,American	L077	0	0	2026-08-20T09:59:13.445+00:...	/Volumes/alankrit_db_wo
7	R007	Pasta Palace Corner	American,Mexican,Chine...	L014	3.3	3.33	2026-08-20T09:59:13.445+00:...	/Volumes/alankrit_db_wo
8	R008	Curry Leaf Grill	Chinese,Mexican	L053	2.7	1.25	2026-08-20T09:59:13.445+00:...	/Volumes/alankrit_db_wo
9	R009	Curry Leaf Corner	Mexican	L010	3.4	3.11	2026-08-20T09:59:13.445+00:...	/Volumes/alankrit_db_wo
10	R010	Pasta Palace Hub	Mexican,Italian,American	L082	3.6	2.76	2026-08-20T09:59:13.445+00:...	/Volumes/alankrit_db_wo

- Creation of Locations Bronze Table and it's output

03:31 PM (10s) 4

```
CREATE OR REPLACE TABLE alan_locations_bronze AS
SELECT
  CAST(location_id AS STRING) AS location_id,
  CAST(area AS STRING) AS area,
  CAST(city AS STRING) AS city,
  CAST(state AS STRING) AS state,
  CAST(latitude AS DOUBLE) AS latitude,
  CAST(longitude AS DOUBLE) AS longitude,
  current_timestamp() AS ingestion_time,
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_locations.
  csv' AS source_file_name
FROM read_files(
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_locations.
  csv',
  format => 'csv',
  header => true,
  inferSchema => true
);
```

(2) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long]

Just now (2s) 7

SQL

```
SELECT * FROM alan_locations_bronze;
```

(1) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [location_id: string, area: string ... 6 more fields]

	location_id	area	city	state	latitude	longitude	ingestion_time	source_file_name
1	L001	Kirk Valley	North Sarah	Michigan	-2.8767365	-62.888285	2026-08-20T10:01:44.649+00:...	/Volumes/alkankrit_db_workspace_7405
2	L002	Cory Vista	New Edgar	Washington	-54.1725655	-46.757126	2026-08-20T10:01:44.649+00:...	/Volumes/alkankrit_db_workspace_7405
3	L003	Daryl Extensions	Port Justinmouth	Michigan	-19.072773	27.938702	2026-08-20T10:01:44.649+00:...	/Volumes/alkankrit_db_workspace_7405
4	L004	Grant Glen	Sellerston	Texas	13.0952805	141.629215	2026-08-20T10:01:44.649+00:...	/Volumes/alkankrit_db_workspace_7405
5	L005	Garcia Neck	Port Allison	Wisconsin	8.239121	26.958565	2026-08-20T10:01:44.649+00:...	/Volumes/alkankrit_db_workspace_7405
6	L006	Smith Rapids	Bobbyport	Missouri	16.931649	-49.121678	2026-08-20T10:01:44.649+00:...	/Volumes/alkankrit_db_workspace_7405
7	L007	Michael Streets	Duranport	West Virginia	-66.185224	-142.948244	2026-08-20T10:01:44.649+00:...	/Volumes/alkankrit_db_workspace_7405
8	L008	John Plains	Lake Kristinton	Oklahoma	-83.706852	28.380354	2026-08-20T10:01:44.649+00:...	/Volumes/alkankrit_db_workspace_7405
9	L009	Dennis Branch	North Brenda	Maryland	15.1168605	-67.825285	2026-08-20T10:01:44.649+00:...	/Volumes/alkankrit_db_workspace_7405
10	L010	Flores Vista	Port Joseph	Louisiana	43.557465	35.970725	2026-08-20T10:01:44.649+00:...	/Volumes/alkankrit_db_workspace_7405

- Creation of Delivery Agents Bronze Table and it's output

03:31 PM (4s) 5

```
CREATE OR REPLACE TABLE alan_delivery_agents_bronze AS
SELECT
  CAST(agent_id AS STRING) AS agent_id,
  CAST(name AS STRING) AS name,
  CAST(phone AS STRING) AS phone,
  CAST(rating AS DOUBLE) AS rating,
  current_timestamp() AS ingestion_time,
  '/Volumes/alkankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/
dim_delivery_agents.csv' AS source_file_name
FROM read_files(
  '/Volumes/alkankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/
dim_delivery_agents.csv',
  format => 'csv',
  header => true,
  inferSchema => true
);
```

(3) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long]

Just now (2s) 9 SQL

```
SELECT * FROM alan_delivery_agents_bronze;
```

(1) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, name: string ... 4 more fields]

	agent_id	name	phone	rating	ingestion_time	source_file_name
1	A001	Lindsey Lang	649.286.1390	3.67	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
2	A002	Heidi Rodriguez	597.686.1508x2345	3.87	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
3	A003	Timothy Dudley	306.628.8953x804	4.57	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
4	A004	Victor Caldwell	null	0	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
5	A005	Rebecca Mullen	998-623-6945x94755	4.47	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
6	A006	William Sullivan	(430)391-1726	3.7	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
7	A007	Courtney Stevens	+1-715-785-0893	4.39	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
8	A008	Amy Patterson	(294)393-3957	4.12	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
9	A009	John Pham	001-319-364-9142x335...	4.91	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
10	A010	Sarah Melendez	444.257.2588	4.18	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii

- Creation of Menu Items Bronze Table and it's output

03:32 PM (4s) 6 SQL

```
CREATE OR REPLACE TABLE alan_menu_items_bronze AS
SELECT
  CAST(item_id AS STRING) AS item_id,
  CAST(restaurant_id AS STRING) AS restaurant_id,
  CAST(item_name AS STRING) AS item_name,
  CAST(price AS DOUBLE) AS price,
  CAST(category AS STRING) AS category,
  current_timestamp() AS ingestion_time,
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_menu_items.
  csv' AS source_file_name
FROM read_files(
  '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/dim_menu_items.
  csv',
  format => 'csv',
  header => true,
  inferSchema => true
);
```

(3) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long]

The screenshot shows a Databricks notebook interface. At the top, there's a status bar with a play button, a checkmark, 'Just now (2s)', a page number '11', and buttons for SQL, a share icon, a full-screen icon, and a menu icon. Below this, a SQL query is entered: `SELECT * FROM alan_menu_items_bronze;`. Underneath the query, it says '(1) Spark Jobs' and a log entry: `> _sqlidf: pyspark.sql.connect.dataframe.DataFrame = [item_id: string, restaurant_id: string ... 5 more fields]`. The main part of the screenshot is a table with 10 rows and 8 columns. The columns are: item_id, restaurant_id, item_name, price, category, ingestion_time, and source_file_name. The table contains 10 records of menu items.

	item_id	restaurant_id	item_name	price	category	ingestion_time	source_file_name
1	I0001	R017	Radio Soda	17.97	Beverage	2026-08-20T10:02:09.494+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan...
2	I0002	R169	Occur Pizza	27.44	Main Course	2026-08-20T10:02:09.494+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan...
3	I0003	R178	Road Nachos	9.5	Appetizer	2026-08-20T10:02:09.494+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan...
4	I0004	R032	Detail Juice	5.98	Beverage	2026-08-20T10:02:09.494+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan...
5	I0005	R042	Floor Nachos	7.2	Appetizer	2026-08-20T10:02:09.494+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan...
6	I0006	R022	Benefit Ice Cream	29.5	Dessert	2026-08-20T10:02:09.494+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan...
7	I0007	R140	Firm Cake	9.29	Dessert	2026-08-20T10:02:09.494+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan...
8	I0008	R077	East Pizza	6.15	Main Course	2026-08-20T10:02:09.494+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan...
9	I0009	R137	Same Brownie	16.12	Dessert	2026-08-20T10:02:09.494+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan...
10	I0010	R134	Wrong Juice	0	Beverage	2026-08-20T10:02:09.494+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan...

Screenshots: The Databricks notebook showing all the CTAS queries and its successful execution output.

3. Creating Filtered Tables:

3. Create cleaned data delta tables

Create following tables from the corresponding bronze tables

customers_filtered

Which contains the records where customer_id, location_id, signup_date, dob, email, name are not missing

restaurants_filtered

Which contains the records where restaurant_id, name, cuisines, location_id, rating, delivery_fee are not missing

locations_filtered

Which contains the records where location_id, area, city, state, latitude, longitude are not missing

delivery_agents_filtered

Which contains the records where agent_id, name, phone, rating are not missing

menu_items_filtered

Which contains the records where item_id, restaurant_id, item_name, price, category are not missing

Solution: Created new Delta tables (suffixed with _filtered) by querying the Bronze tables. Applied strict IS NOT NULL conditions across all critical business columns as mandated by the project requirements.

****Attached all the screenshots for the same below****

- Creation of Customers Filtered Table, it's output and comparison with it's Bronze Table

▶ ✓ 21 hours ago (5s)

```
CREATE OR REPLACE TABLE alan_customers_filtered AS
SELECT * FROM alan_customers_bronze
WHERE customer_id IS NOT NULL
  AND location_id IS NOT NULL
  AND signup_date IS NOT NULL
  AND dob IS NOT NULL
  AND email IS NOT NULL
  AND name IS NOT NULL;
```

▶ Just now (2s) 13 SQL

SELECT * FROM alan_customers_filtered;

▶ (1) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, name: string ... 6 more fields]

	customer_id	name	email	dob	signup_date	location_id	ingestion_time	source_file_name
1	C0001	Matthew Rodriguez	heatherdouglas@example.org	1954-12-02	2019-05-16	L069	2026-08-20T09:57:35.645+00...	/Volumes/alkrit...
2	C0002	Justin Diaz	millernathan@example.net	1985-07-05	2018-09-06	L016	2026-08-20T09:57:35.645+00...	/Volumes/alkrit...
3	C0003	Patricia Pena	kristinesanchez@example.com	2003-10-19	2012-05-21	L049	2026-08-20T09:57:35.645+00...	/Volumes/alkrit...
4	C0004	David Smith	cassandra07@example.org	1993-02-22	2003-03-29	L011	2026-08-20T09:57:35.645+00...	/Volumes/alkrit...
5	C0005	Meghan George	johnsonrandall@example.com	1956-05-17	2016-01-27	L071	2026-08-20T09:57:35.645+00...	/Volumes/alkrit...
6	C0006	Jennifer Beard	djohnson@example.net	1974-02-12	1991-01-17	L038	2026-08-20T09:57:35.645+00...	/Volumes/alkrit...
7	C0007	Tiffany Davis	thomasjoseph@example.com	1978-05-20	1998-04-16	L081	2026-08-20T09:57:35.645+00...	/Volumes/alkrit...
8	C0008	Rachel Elliott	pbarnes@example.com	1964-07-20	1997-04-21	L080	2026-08-20T09:57:35.645+00...	/Volumes/alkrit...
9	C0009	Mark Martin	ijohnson@example.org	1964-05-01	1980-04-16	L047	2026-08-20T09:57:35.645+00...	/Volumes/alkrit...
10	C0010	Julie Murphy	amy49@example.net	1968-05-28	2014-11-27	L074	2026-08-20T09:57:35.645+00...	/Volumes/alkrit...

▶ ✓ 21 hours ago (4s)

```
SELECT 'Bronze' AS layer, COUNT(*) AS count FROM alan_customers_bronze
UNION ALL
SELECT 'Filtered' AS layer, COUNT(*) AS count FROM alan_customers_filtered;
```

▶ (4) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [layer: string, count: long]

	layer	count
1	Bronze	10050
2	Filtered	9999

- Creation of Restaurants Filtered Table, it's output and comparison with it's Bronze Table

```

▶ 21 hours ago (3s)

CREATE OR REPLACE TABLE alan_restaurants_filtered AS
SELECT * FROM alan_restaurants_bronze
WHERE restaurant_id IS NOT NULL
    AND name IS NOT NULL
    AND cuisines IS NOT NULL
    AND location_id IS NOT NULL
    AND rating IS NOT NULL
    AND delivery_fee IS NOT NULL;

```

Just now (6s) 16 SQL

SELECT * FROM alan_restaurants_filtered;

▶ (1) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: string, name: string ... 6 more fields]

Table	+								
restaurant_id	name	cuisines	location_id	rating	delivery_fee	ingestion_time	source_file_name		
1	R001	Food Haven Grill	Indian	L069	3.3	3.9	2026-08-20T09:59:13.445+00:...	/Volumes/alkankrit_db_wo	
2	R002	Spice Villa Hub	Thai,Italian,Chinese	L012	3.8	2.25	2026-08-20T09:59:13.445+00:...	/Volumes/alkankrit_db_wo	
3	R004	Curry Leaf Hub	Indian,American	L048	3.1	3.03	2026-08-20T09:59:13.445+00:...	/Volumes/alkankrit_db_wo	
4	R005	Tasty Bites Grill	Mexican	L039	3.9	1.02	2026-08-20T09:59:13.445+00:...	/Volumes/alkankrit_db_wo	
5	R007	Pasta Palace Corner	American,Mexican,Chine...	L014	3.3	3.33	2026-08-20T09:59:13.445+00:...	/Volumes/alkankrit_db_wo	
6	R008	Curry Leaf Grill	Chinese,Mexican	L053	2.7	1.25	2026-08-20T09:59:13.445+00:...	/Volumes/alkankrit_db_wo	
7	R009	Curry Leaf Corner	Mexican	L010	3.4	3.11	2026-08-20T09:59:13.445+00:...	/Volumes/alkankrit_db_wo	
8	R010	Pasta Palace Hub	Mexican,Italian,American	L082	3.6	2.76	2026-08-20T09:59:13.445+00:...	/Volumes/alkankrit_db_wo	
9	R011	Curry Leaf Corner	Chinese,Thai	L017	3.7	2.83	2026-08-20T09:59:13.445+00:...	/Volumes/alkankrit_db_wo	
10	R012	Pasta Palace Hub	Indian,Chinese,Mexican	L061	2.6	1.63	2026-08-20T09:59:13.445+00:...	/Volumes/alkankrit_db_wo	

▶ 21 hours ago (2s) 17

```

SELECT 'Bronze' AS layer, COUNT(*) AS count FROM alan_restaurants_bronze
UNION ALL
SELECT 'Filtered' AS layer, COUNT(*) AS count FROM alan_restaurants_filtered;

```

▶ (4) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [layer: string, count: long]

Table	+		
layer	count		
1	Bronze	200	
2	Filtered	150	

- Creation of Locations Filtered Table, it's output and comparison with it's Bronze Table

```

▶ ✓ 21 hours ago (3s)

CREATE OR REPLACE TABLE alan_locations_filtered AS
SELECT * FROM alan_locations_bronze
WHERE location_id IS NOT NULL
    AND area IS NOT NULL
    AND city IS NOT NULL
    AND state IS NOT NULL
    AND latitude IS NOT NULL
    AND longitude IS NOT NULL;

```

Just now (2s) 19 SQL

SELECT * FROM alan_locations_filtered;

(1) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [location_id: string, area: string ... 6 more fields]

Table +

	location_id	area	city	state	latitude	longitude	ingestion_time	source_file_name
1	L001	Kirk Valley	North Sarah	Michigan	-2.8767365	-62.888285	2026-08-20T10:01:44.649+00:...	/Volumes/alankrit_db_workspace_7405
2	L002	Cory Vista	New Edgar	Washington	-54.1725655	-46.757126	2026-08-20T10:01:44.649+00:...	/Volumes/alankrit_db_workspace_7405
3	L003	Daryl Extensions	Port Justinmouth	Michigan	-19.072773	27.938702	2026-08-20T10:01:44.649+00:...	/Volumes/alankrit_db_workspace_7405
4	L004	Grant Glen	Sellerston	Texas	13.0952805	141.629215	2026-08-20T10:01:44.649+00:...	/Volumes/alankrit_db_workspace_7405
5	L005	Garcia Neck	Port Allison	Wisconsin	8.239121	26.958565	2026-08-20T10:01:44.649+00:...	/Volumes/alankrit_db_workspace_7405
6	L006	Smith Rapids	Bobbyport	Missouri	16.931649	-49.121678	2026-08-20T10:01:44.649+00:...	/Volumes/alankrit_db_workspace_7405
7	L007	Michael Streets	Duranport	West Virginia	-66.185224	-142.948244	2026-08-20T10:01:44.649+00:...	/Volumes/alankrit_db_workspace_7405
8	L008	John Plains	Lake Kristinton	Oklahoma	-83.706852	28.380354	2026-08-20T10:01:44.649+00:...	/Volumes/alankrit_db_workspace_7405
9	L009	Dennis Branch	North Brenda	Maryland	15.1168605	-67.825285	2026-08-20T10:01:44.649+00:...	/Volumes/alankrit_db_workspace_7405
10	L010	Flores Vista	Port Joseph	Louisiana	43.557465	35.970725	2026-08-20T10:01:44.649+00:...	/Volumes/alankrit_db_workspace_7405

21 hours ago (2s) 20

```

SELECT 'Bronze' AS layer, COUNT(*) AS count FROM alan_locations_bronze
UNION ALL
SELECT 'Filtered' AS layer, COUNT(*) AS count FROM alan_locations_filtered;

```

(4) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [layer: string, count: long]

Table +

	layer	count
1	Bronze	100
2	Filtered	100

- Creation of Delivery Agents Filtered Table, it's output and comparison with it's Bronze Table

```

▶ 21 hours ago (3s)

CREATE OR REPLACE TABLE alan_delivery_agents_filtered AS
SELECT * FROM alan_delivery_agents_bronze
WHERE agent_id IS NOT NULL
      AND name IS NOT NULL
      AND phone IS NOT NULL
      AND rating IS NOT NULL;

```

Just now (1s) 22

SQL

SELECT * FROM alan_delivery_agents_filtered;

(1) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, name: string ... 4 more fields]

	agent_id	name	phone	rating	ingestion_time	source_file_name
1	A001	Lindsey Lang	649.286.1390	3.67	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
2	A002	Heidi Rodriguez	597.686.1508x2345	3.87	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
3	A003	Timothy Dudley	306.628.8953x804	4.57	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
4	A005	Rebecca Mullen	998-623-6945x94755	4.47	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
5	A006	William Sullivan	(430)391-1726	3.7	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
6	A007	Courtney Stevens	+1-715-785-0893	4.39	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
7	A008	Amy Patterson	(294)393-3957	4.12	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
8	A009	John Pham	001-319-364-9142x335...	4.91	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
9	A010	Sarah Melendez	444.257.2588	4.18	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii
10	A011	Erin Garcia	001-975-458-5633x2073	4.81	2026-08-20T10:02:01.220+00:...	/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dii

21 hours ago (2s) 23

```

SELECT 'Bronze' AS layer, COUNT(*) AS count FROM alan_delivery_agents_bronze
UNION ALL
SELECT 'Filtered' AS layer, COUNT(*) AS count FROM alan_delivery_agents_filtered;

```

(4) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [layer: string, count: long]

	layer	count
1	Bronze	450
2	Filtered	395

- Creation of Menu Items Filtered Table, it's output and comparison with it's Bronze Table

```

▶ 21 hours ago (3s)

CREATE OR REPLACE TABLE alan_menu_items_filtered AS
SELECT * FROM alan_menu_items_bronze
WHERE item_id IS NOT NULL
    AND restaurant_id IS NOT NULL
    AND item_name IS NOT NULL
    AND price IS NOT NULL
    AND category IS NOT NULL;

```

Just now (1s) 25

SQL

SELECT * FROM alan_menu_items_filtered;

▶ (1) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [item_id: string, restaurant_id: string ... 5 more fields]

Table +

	item_id	restaurant_id	item_name	price	category	ingestion_time	source_file_name
1	I0001	R017	Radio Soda	17.97	Beverage	2026-08-20T10:02:09.494+00:...	/Volumes/alkrit_db_workspace_7405609522919643/alan...
2	I0002	R169	Occur Pizza	27.44	Main Course	2026-08-20T10:02:09.494+00:...	/Volumes/alkrit_db_workspace_7405609522919643/alan...
3	I0003	R178	Road Nachos	9.5	Appetizer	2026-08-20T10:02:09.494+00:...	/Volumes/alkrit_db_workspace_7405609522919643/alan...
4	I0004	R032	Detail Juice	5.98	Beverage	2026-08-20T10:02:09.494+00:...	/Volumes/alkrit_db_workspace_7405609522919643/alan...
5	I0005	R042	Floor Nachos	7.2	Appetizer	2026-08-20T10:02:09.494+00:...	/Volumes/alkrit_db_workspace_7405609522919643/alan...
6	I0006	R022	Benefit Ice Cream	29.5	Dessert	2026-08-20T10:02:09.494+00:...	/Volumes/alkrit_db_workspace_7405609522919643/alan...
7	I0007	R140	Firm Cake	9.29	Dessert	2026-08-20T10:02:09.494+00:...	/Volumes/alkrit_db_workspace_7405609522919643/alan...
8	I0008	R077	East Pizza	6.15	Main Course	2026-08-20T10:02:09.494+00:...	/Volumes/alkrit_db_workspace_7405609522919643/alan...
9	I0009	R137	Same Brownie	16.12	Dessert	2026-08-20T10:02:09.494+00:...	/Volumes/alkrit_db_workspace_7405609522919643/alan...
10	I0010	R134	Wrong Juice	0	Beverage	2026-08-20T10:02:09.494+00:...	/Volumes/alkrit_db_workspace_7405609522919643/alan...

21 hours ago (2s) 26

```

SELECT 'Bronze' AS layer, COUNT(*) AS count FROM alan_menu_items_bronze
UNION ALL
SELECT 'Filtered' AS layer, COUNT(*) AS count FROM alan_menu_items_filtered;

```

▶ (4) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [layer: string, count: long]

Table +

	layer	count
1	Bronze	500
2	Filtered	500

Screenshots: A side-by-side SQL query result showing the row count difference between a Bronze table and its corresponding filtered table.

4. Creating orders_bronze Table using COPY INTO:

Incremental Data Loading

4. Create a table named `orders_bronze` without schema and incrementally load order details for each month of the year 2024, ingesting one month's data at a time, using `COPY INTO`

Ingest January, February month order details, verify the records count and then ingest March month order details, verify the records count

Solution:

Created a schemaless Delta table (`alan_orders_bronze`) and executed the `COPY INTO` command to ingest the JSON files from the Unity Catalog Volume. Configured the command with `inferSchema = 'true'` and `mergeSchema = 'true'` to allow Databricks to dynamically adapt to the JSON structure.

****Attached all the screenshots for the same below****

- Using `COPY INTO` to ingest January and February Data into `alan_orders_bronze` Table

The screenshot shows the Databricks SQL interface. At the top, a status bar indicates a successful execution at 04:17 PM (7s) with a progress indicator of 17. Below this, the SQL command is displayed:

```
CREATE TABLE IF NOT EXISTS alan_orders_bronze;

COPY INTO alan_orders_bronze
FROM '/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw/orders'
FILEFORMAT = JSON
FORMAT_OPTIONS ('inferSchema' = 'true')
COPY_OPTIONS ('mergeSchema' = 'true');
```

Below the command, it shows "(2) Spark Jobs". Underneath, a message indicates the command was successful: `_sqlcdf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows: long ... 1 more field]`.

At the bottom, a table view is shown with the following data:

	1^2_3 num_affected_rows	1^2_3 num_inserted_rows	1^2_3 num_skipped_corrupt_files
1	11632	11632	0

- Output of `alan_orders_bronze` table data:

Just now (2s) 28 SQL

```
SELECT * FROM alan_orders_bronze;
```

(1) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, customer_id: string ... 9 more fields]

	agent_id	customer_id	delivery_location_id	items_ordered
1	A052	C7420	L049	> [{"item_id":"I0117","item_name":"Over Juice","price":0,"quantity":5}]
2	A270	C9458	L020	> [{"item_id":"I0329","item_name":"Job Ice Cream","price":6.66,"quantity":3}, {"item_id":"I0251","item_name":"Follow Tea","price":2
3	A388	C5695	L025	> [{"item_id":"I0052","item_name":"To Nachos","price":28.81,"quantity":3}]
4	A018	C5784	L093	> [{"item_id":"I0228","item_name":"Laugh Juice","price":6.07,"quantity":1}, {"item_id":"I0097","item_name":"Attack Biryani","price":
5	A279	C2581	L052	> [{"item_id":"I0255","item_name":"Finish Garlic Bread","price":0,"quantity":3}]
6	A099	C0733	L060	> [{"item_id":"I0382","item_name":"Magazine Pasta","price":27.07,"quantity":5}, {"item_id":"I0443","item_name":"He Coffee","price
7	A310	C1979	L084	> [{"item_id":"I0113","item_name":"Have Tea","price":15.94,"quantity":2}]
8	A393	C1523	L007	> [{"item_id":"I0495","item_name":"Operation Tea","price":22.44,"quantity":1}, {"item_id":"I0149","item_name":"Wide Brownie","pri
9	A008	C0373	L086	> [{"item_id":"I0469","item_name":"Change Ice Cream","price":13.1,"quantity":2}, {"item_id":"I0352","item_name":"Movie Juice","pr
10	A206	C5037	L088	> [{"item_id":"I0462","item_name":"Their Garlic Bread","price":26.4,"quantity":2}]

4 minutes ago (2s) 18

```
SELECT COUNT(*) AS total_orders_loaded FROM alan_orders_bronze;
```

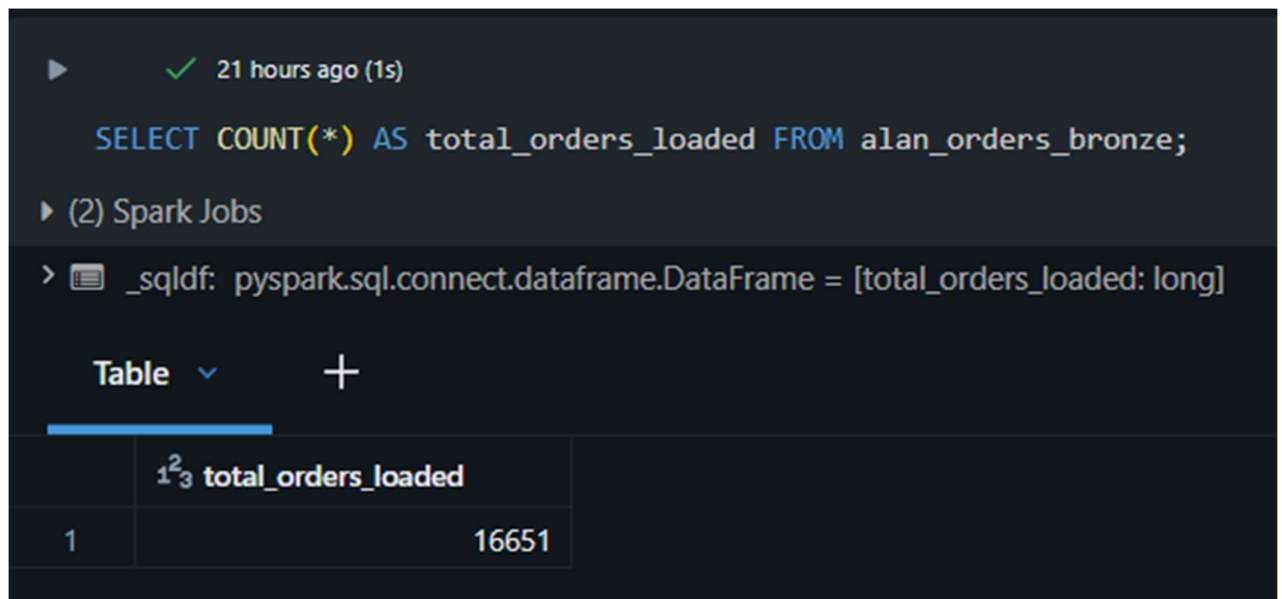
(3) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [total_orders_loaded: long]

	total_orders_loaded
1	11632

Screenshots: The SQL execution output of the COPY INTO command showing the number of files processed and rows inserted from the January and February Data.

- Simulated a new data delivery by uploading **orders_2024_03.json** to the volume. Re-ran the identical **COPY INTO** command without altering the SQL syntax.



Screenshot: The output of SELECT COUNT(*) showing the updated 16,651 row count after uploading March data.

Problem Statements Orders Data Analysis:

Problem Statements

NOTE: All numeric calculations should be rounded UP to two decimal places for floating-point numbers and one decimal place for discrete entities.

Price mentioned is in dollars(\$)

Orders Data Analysis

Write Databricks SQL query to find

Solution:

Executed a series of aggregation queries directly on the alan_orders_bronze table. Implemented explicit rounding logic (CEIL(value * 100) / 100) to fulfill the business rule requiring all floating-point metrics to be rounded up to two decimal places. Used the EXPLODE() function to flatten the JSON array items_ordered for item-level analysis.

****Attached all the screenshots for the same below****

The answers to the queries asked are as follows:

1. Total number of orders placed

04:38 PM (2s) 1

```
SELECT
  COUNT(order_id) AS total_orders
FROM alan_orders_bronze;
```

(2) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame [long]

	total_orders
1	16651

2. Distribution of orders by payment method

04:38 PM (2s) 2

```
SELECT
  payment_method,
  COUNT(order_id) AS total_orders
FROM alan_orders_bronze
GROUP BY payment_method
ORDER BY total_orders DESC;
```

(2) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [payment_method: string, total_orders: long]

	payment_method	total_orders
1	cash	4201
2	card	4186
3	wallet	4140
4	UPI	4124

3. Distribution of order status (e.g., delivered, preparing, cancelled)

04:38 PM (2s) 3

```
SELECT
  order_status,
  COUNT(order_id) AS total_orders
FROM alan_orders_bronze
GROUP BY order_status
ORDER BY total_orders DESC;
```

(2) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame [string, total_orders: long]

	order_status	total_orders
1	en route	4238
2	preparing	4179
3	cancelled	4129
4	delivered	4105

4. Orders per restaurant (volume of business)

▶ 05:16 PM (2s)

```
SELECT
  restaurant_id,
  COUNT(order_id) AS total_orders
FROM alan_orders_bronze
GROUP BY restaurant_id
ORDER BY total_orders DESC;
```

▶ (2) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame

Table ▾ +

	^A _C restaurant_id	¹ ₂ ³ total_orders
1	R005	114
2	R198	111
3	R031	110
4	R131	108
5	R083	108

5. Orders per customer (loyalty insight)

▶ 05:16 PM (1s)

```
SELECT
  customer_id,
  COUNT(order_id) AS total_orders
FROM alan_orders_bronze
GROUP BY customer_id
ORDER BY total_orders DESC;
```

▶ (2) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame

Table ▾ +

	^A _C customer_id	¹ ₂ ³ total_orders
1	C7381	9
2	C6444	8
3	C0362	8
4	C2231	8
5	C6672	8

6. Orders handled per delivery agent

05:16 PM (1s)

```
SELECT
  agent_id,
  COUNT(order_id) AS total_orders
FROM alan_orders_bronze
GROUP BY agent_id
ORDER BY total_orders DESC;
```

(2) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame

Table +

	^A _C agent_id	¹ ₂ ³ total_orders
1	A309	117
2	A207	114
3	A393	99
4	A350	91
5	A138	88
6	A245	86

7. Top 10 orders by total amount

04:40 PM (1s) 7

```
SELECT
  order_id,
  customer_id,
  restaurant_id,
  total_amount,
  payment_method,
  order_status
FROM alan_orders_bronze
ORDER BY total_amount DESC
LIMIT 10;
```

(1) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [order_id: string, customer_id: string ... 4 more fields]

Table +

	^A _C order_id	^A _C customer_id	^A _C restaurant_id	¹ ₂ total_amount	^A _C payment_method
1	c12860c3-683a-47f7-b790-7cbbbcfc0fec	C5726	R159	361.46	cash
2	05e56a7c-190c-4bde-ac13-e34b38fe155a	C5959	R151	355.9	cash
3	341cb220-46e4-4f0d-b2f2-2e494b5729dd	C6763	R147	352.45	card
4	3b6f5bd2-08e4-412d-be9e-7701463015...	C3608	R105	348.95	card
5	f43228bd-94e5-4ef6-914a-c7ebdceaa572	C4740	R056	342.91	wallet

8. Average tip amount per payment method

▶ 05:10 PM (2s)

```
SELECT
  payment_method,
  CEIL(AVG(tip) * 100) / 100 AS avg_tip_amount
FROM alan_orders_bronze
GROUP BY payment_method;
```

▶ (2) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [payment_method, avg_tip_amount]

Table +

	payment_method	avg_tip_amount
1	wallet	2.5
2	cash	2.48
3	card	2.51
4	UPI	2.53

9. Percentage of orders with tip > 10% of total_amount

▶ Just now (3s)

```
SELECT
  CAST(CEIL((COUNT(
    CASE
      WHEN tip > 0.10 * total_amount THEN 1
    END
  ) * 100.0 / COUNT(*)) * 100
  ) / 100
  AS DECIMAL(10,2)
  ) AS percentage_high_tip
FROM alan_orders_bronze;
```

▶ (2) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [percentage_high_tip]

Table +

	percentage_high_tip
1	21.49

10. Percentage of successful vs cancelled vs failed orders.

Just now (3s)

```

SELECT
  order_status,
  CAST(CEIL(
    COUNT(*) * 100.0 /
    (SELECT COUNT(*)
     FROM alan_orders_bronze)
    * 100
  ) / 100
  AS DECIMAL(10,2)
) AS percentage
FROM alan_orders_bronze
GROUP BY order_status
ORDER BY order_status;

```

► (5) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame

Table	+
^A _C order_status	.00 percentage
1 cancelled	24.80
2 delivered	24.66
3 en route	25.46
4 preparing	25.10

11. Top customers (by number of orders) and top restaurants (by number of orders).

a)

05:15 PM (2s)

```

SELECT
  customer_id,
  COUNT(order_id) AS total_orders
FROM alan_orders_bronze
GROUP BY customer_id
ORDER BY total_orders DESC
LIMIT 5;

```

► (2) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame

Table	+
^A _C customer_id	¹ ₂ ³ total_orders
1 C7381	9
2 C6444	8
3 C2231	8
4 C6672	8
5 C0362	8

b)

05:14 PM (1s)

```

SELECT
  restaurant_id,
  COUNT(order_id) AS total_orders
FROM alan_orders_bronze
GROUP BY restaurant_id
ORDER BY total_orders DESC
LIMIT 5;

```

► (2) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame

Table	+
^A _C restaurant_id	¹ ₂ ³ total_orders
1 R005	114
2 R198	111
3 R031	110
4 R171	108
5 R083	108

12. Item-Level Insights (explode items_ordered)

- Find most frequently ordered items
- Find average number of items per order
- Find item revenue contribution per restaurant

a)

▶

✓

05:13 PM (2s)

```
SELECT
  item.item_name,
  COUNT(*) AS frequency
FROM (
  SELECT EXPLODE(items_ordered)
  AS item FROM alan_orders_bronze
)
GROUP BY item.item_name
ORDER BY frequency DESC
LIMIT 10;
```

▶ (2) Spark Jobs

>  _sqldf: pyspark.sql.connect.dataframe.DataFrame

Table ▼

+

	^A _c item_name	¹ ² ₃ frequency
1	Cell Garlic Bread	161
2	Magazine Pasta	152
3	Painting Coffee	151
4	Tough Garlic Bread	138
5	Section Coffee	132
6	Huge Nachos	129
7	Argue Cake	111
8	Have Tea	107
9	...	...

b)

05:14 PM (2s)

```

SELECT
  CEIL(AVG(SIZE(items_ordered))
    * 100) / 100
  AS avg_items_per_order
FROM alan_orders_bronze;

```

▶ (2) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.Da

Table ▾ +

	1.2 avg_items_per_order
1	1.65

c)

Just now (3s) 15

```

SELECT
  restaurant_id,
  item.item_name,
  CEIL(SUM(item.price * item.quantity) * 100) / 100 AS item_revenue
FROM (
  SELECT
    restaurant_id,
    EXPLODE(items_ordered) AS item
  FROM alan_orders_bronze
)
GROUP BY
  restaurant_id,
  item.item_name
ORDER BY
  restaurant_id,
  item_revenue DESC;

```

▶ (2) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [restaurant_id: string, item_name: string

Table ▾ +

	A ^B _C restaurant_id	A ^B _C item_name	1.2 item_revenue
1	R001	Section Coffee	3641.8
2	R001	Type Ice Cream	2556.05
3	R001	Theory Spring Roll	1107
4	R001	Question Biryani	980.13
5	R001	Sort Brownie	964.17

Orders Data and Dimension Datasets Analysis

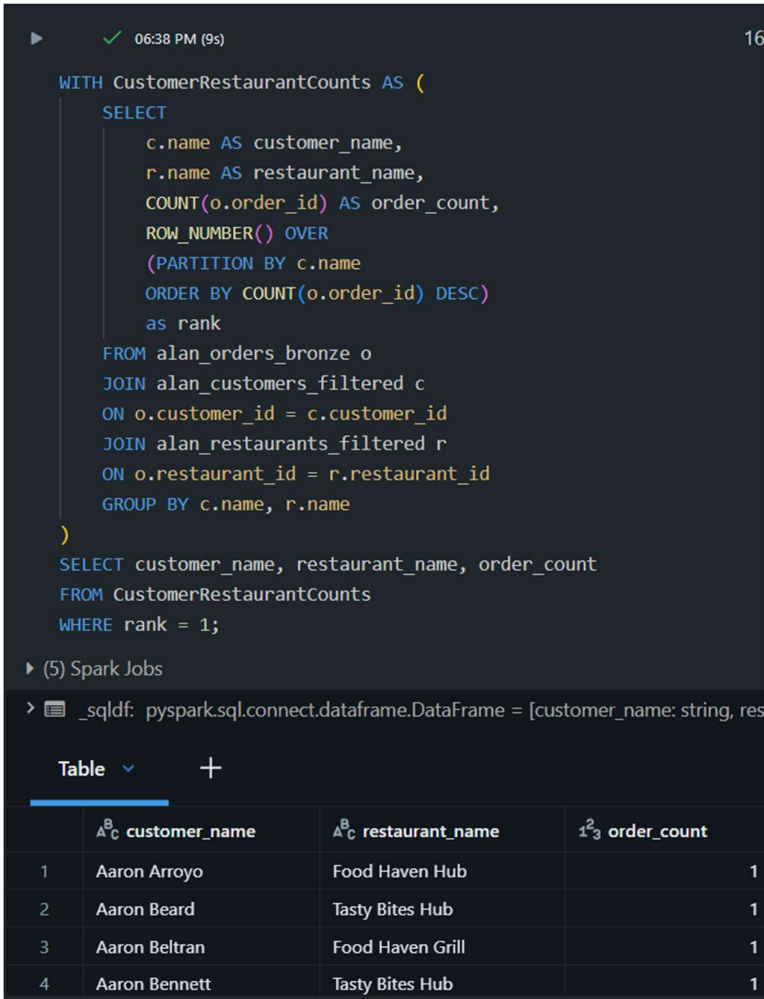
Solution:

Authored analytical SQL queries applying window functions (ROW_NUMBER() OVER PARTITION BY), date/time parsing, and multi-table inner joins. Handled zero-division edge cases gracefully by leveraging try_divide.

****Attached all the screenshots for the same below****

The answers to the queries asked are as follows:

1. Find the preferred restaurant names for each customer



```
WITH CustomerRestaurantCounts AS (
  SELECT
    c.name AS customer_name,
    r.name AS restaurant_name,
    COUNT(o.order_id) AS order_count,
    ROW_NUMBER() OVER
      (PARTITION BY c.name
       ORDER BY COUNT(o.order_id) DESC)
      AS rank
  FROM alan_orders_bronze o
  JOIN alan_customers_filtered c
  ON o.customer_id = c.customer_id
  JOIN alan_restaurants_filtered r
  ON o.restaurant_id = r.restaurant_id
  GROUP BY c.name, r.name
)
SELECT customer_name, restaurant_name, order_count
FROM CustomerRestaurantCounts
WHERE rank = 1;
```

▶ (5) Spark Jobs

> `_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [customer_name: string, res`

	customer_name	restaurant_name	order_count
1	Aaron Arroyo	Food Haven Hub	1
2	Aaron Beard	Tasty Bites Hub	1
3	Aaron Beltran	Food Haven Grill	1
4	Aaron Bennett	Tasty Bites Hub	1

2. Find the top 10 areas receiving orders in each month

06:38 PM (2s)

```
WITH AreaMonthlyCounts AS (  
  SELECT  
    MONTH(CAST(o.timestamp AS TIMESTAMP))  
    AS order_month,  
    l.area AS area_name,  
    COUNT(o.order_id) AS total_orders,  
    ROW_NUMBER() OVER  
      (PARTITION BY MONTH(CAST(o.timestamp  
        AS TIMESTAMP))  
       ORDER BY COUNT(o.order_id) DESC)  
      AS rank  
    FROM alan_orders_bronze o  
    JOIN alan_locations_filtered l  
    ON o.delivery_location_id = l.location_id  
    GROUP BY MONTH(CAST(o.timestamp AS TIMESTAMP)), l.area  
  )  
SELECT order_month, area_name, total_orders  
FROM AreaMonthlyCounts  
WHERE rank <= 10  
ORDER BY order_month, rank;
```

▶ (4) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [order_month: integer]

Table +

	1 ² order_month	A ^B area_name	1 ² total_orders
1	1	Rodriguez Mill	87
2	1	Eric Hills	86
3	1	Hill Station	80
4	1	Smith Estates	79

3. Find all successfully delivered orders that were paid using a card, to analyze payment preferences and delivery success rates

06:47 PM (2s) 18 SQL

```
SELECT *  
FROM alan_orders_bronze  
WHERE LOWER(payment_method) = 'card'  
AND LOWER(order_status) = 'delivered';
```

▶ (1) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, customer_id: string ... 9 more fields]

Table +

	A ^B agent_id	A ^B customer_id	A ^B delivery_location_id	items_ordered
1	A086	C5605	L089	> [{"item_id": "I0491", "item_name": "Talk Pasta", "price": 10.6, "quantity": 2}]
2	A352	C7525	L082	> [{"item_id": "I0403", "item_name": "Outside Juice", "price": 23.61, "quantity": 1}, {"item_id": "I00
3	A068	C1909	L083	> [{"item_id": "I0038", "item_name": "Near Soda", "price": 15.97, "quantity": 4}, {"item_id": "I0134"

4. Which cuisines generate the highest average order value?

Hint: Join orders ,restaurants, group by cuisine, and calculate average total_amount.

06:38 PM (2s)

```
SELECT
  r.cuisines,
  CEIL(AVG(o.total_amount) * 100)
  / 100 AS avg_order_value
FROM alan_orders_bronze o
JOIN alan_restaurants_filtered r
ON o.restaurant_id = r.restaurant_id
GROUP BY r.cuisines
ORDER BY avg_order_value DESC;
```

▶ (3) Spark Jobs

>  _sqlidf: pyspark.sql.connect.dataframe.DataFrame = [cuis

Table  

	^A _C cuisines	1.2 avg_order_value
1	Thai,Chinese	160.11
2	Thai,Italian,Chinese	156.69
3	Mexican,American	146.19
4	Mexican,Indian,Thai	144.91
5	Mexican,American,Thai	143.95

5. Which delivery agents have delivered the most high-value orders (above \$50)?

Hint: Filter orders for total_amount > 50, join with agent, group by agent_id, count orders.

06:38 PM (2s)

```

SELECT
  a.name AS agent_name,
  COUNT(o.order_id)
  AS high_value_orders
FROM alan_orders_bronze o
JOIN alan_delivery_agents_filtered a
ON o.agent_id = a.agent_id
WHERE o.total_amount > 50
GROUP BY a.name
ORDER BY high_value_orders DESC;

```

▶ (3) Spark Jobs

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [a`

Table ▼ +

	^A _C agent_name	¹ ₃ high_value_orders
1	Christopher Chase	213
2	Christopher Williams	171
3	Justin Lam	118
4	Charlene Morgan	112
5	Joan Harrison	110

6. Which cities (area_name) are generating the most revenue for Italian restaurants?

Hint: Join orders , restaurants , locations, filter cuisine = 'Italian', group by area_name, sum total amount

06:53 PM (2s)

```
SELECT
  l.area,
  CEIL(SUM(o.total_amount) * 100)
  / 100
  AS total_revenue
FROM alan_orders_bronze o
JOIN alan_restaurants_filtered r
ON o.restaurant_id = r.restaurant_id
JOIN alan_locations_filtered l
ON r.location_id = l.location_id
WHERE r.cuisines = 'Italian'
GROUP BY l.area
ORDER BY total_revenue DESC;
```

▶ (4) Spark Jobs

▶ `_sqldf: pyspark.sql.connect.dataframe.DataFrame`

Table

+

	^A _C area	1.2 total_revenue
1	Henderson Cre...	11356.8
2	Burns Burg	4822.74
3	Williams Way	3168.24

7. What is the tip-to-total ratio by cuisine type?

Hint: Join orders , restaurants, group by cuisine, calculate average tip / total_amount.

06:53 PM (1s)

```

SELECT
  r.cuisines,
  CEIL(AVG(try_divide(o.tip, o.total_amount))
    * 100)
  / 100
  AS tip_to_total_ratio
FROM alan_orders_bronze o
JOIN alan_restaurants_filtered r
ON o.restaurant_id = r.restaurant_id
GROUP BY r.cuisines
ORDER BY tip_to_total_ratio DESC;

```

▶ (3) Spark Jobs

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [cuisi`

Table ▼ +

	^A _C cuisines	1.2 tip_to_total_ratio
1	Indian,Mexican,American	0.22
2	Thai,American,Indian	0.17
3	Chinese,Thai	0.16
4	Chinese,American	0.11
5	Italian,Thai,Chinese	0.1

8. Which menu items are most frequently ordered, and from which restaurants?

Hint: Explode `items_ordered` in `orders`, join with `menu_items`, group by `item_name`, `restaurant_id`, count.

▶

06:53 PM (2s)

23

```

SELECT
  i.item_name,
  r.name AS restaurant_name,
  COUNT(*) AS order_frequency
FROM (
  SELECT restaurant_id, EXPLODE(items_ordered)
  AS item FROM alan_orders_bronze
) exploded_orders
JOIN alan_menu_items_filtered i
ON exploded_orders.item.item_id = i.item_id
JOIN alan_restaurants_filtered r
ON exploded_orders.restaurant_id = r.restaurant_id
GROUP BY i.item_name, r.name
ORDER BY order_frequency DESC
LIMIT 10;

```

▶ (4) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [item_name: string, restaurant_name: string, order_frequency: int]

Table ▼

+

	^A _C item_name	^A _C restaurant_name	¹ ₃ order_frequency
1	Argue Cake	Curry Leaf Kitchen	111
2	Even Tea	Food Haven Kitchen	103
3	Rich Biryani	Curry Leaf Corner	101
4	Sell Ice Cream	Food Haven Grill	101
5	Finish Garlic Bread	Spice Villa Hub	100

9. Which agents have delivered the most diverse cuisines?

Hint: Join orders , restaurants, group by agent_id, count distinct cuisine.

06:54 PM (2s)

```

SELECT
  a.name AS agent_name,
  COUNT(DISTINCT r.cuisines)
  AS distinct_cuisines_delivered
FROM alan_orders_bronze o
JOIN alan_restaurants_filtered r
ON o.restaurant_id = r.restaurant_id
JOIN alan_delivery_agents_filtered a
ON o.agent_id = a.agent_id
GROUP BY a.name
ORDER BY distinct_cuisines_delivered DESC;

```

▶ (5) Spark Jobs

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [agent_name, distinct_cuisines_delivered]

Table

	^A _C agent_name	¹ ₂ ³ distinct_cuisines_delivered
1	Christopher Chase	43
2	Justin Smith	40
3	Justin Lam	39
4	Christopher Williams	39
5	James Osborn	38

10. What is the average customer lifetime value by signup year?

Hint: Join orders , customers, extract year from signup_date, group by year, sum total_amount per customer, then average total_amount per customer, then average

06:54 PM (2s)

```

WITH CustomerSpend AS (
  SELECT
    YEAR(CAST(c.signup_date AS DATE))
    AS signup_year,
    c.customer_id,
    SUM(o.total_amount)
    AS total_spend
  FROM alan_orders_bronze o
  JOIN alan_customers_filtered c
  ON o.customer_id = c.customer_id
  GROUP BY YEAR(CAST(c.signup_date
  AS DATE)), c.customer_id
)
SELECT
  signup_year,
  CEIL(AVG(total_spend)
  * 100) / 100
  AS avg_customer_ltv
FROM CustomerSpend
GROUP BY signup_year
ORDER BY signup_year;

```

► (4) Spark Jobs

► _sqldf: pyspark.sql.connect.dataframe.DataFrame = [s

Table

	1.3 signup_year	1.2 avg_customer_ltv
1	1954	22.64
2	1955	172.65
3	1956	139.91

11. Which restaurants have the highest order frequency per location?

Hint: Join orders , restaurants , locations, group by restaurant_id, location_id, count orders.

▶

06:56 PM (2s)

26

```

SELECT
  r.name AS restaurant_name,
  l.area AS location_area,
  COUNT(o.order_id) AS order_frequency
FROM alan_orders_bronze o
JOIN alan_restaurants_filtered r
ON o.restaurant_id = r.restaurant_id
JOIN alan_locations_filtered l
ON o.delivery_location_id = l.location_id
GROUP BY r.name, l.area
ORDER BY order_frequency DESC;

```

▶ (4) Spark Jobs

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [restaurant_name: string, locati`

Table

+

	^A _C restaurant_name	^A _C location_area	¹ ₃ order_frequency
1	Spice Villa Grill	Joshua Drive	21
2	Curry Leaf Corner	Carpenter Views	21
3	Spice Villa Hub	Rodriguez Mill	19
4	Curry Leaf Corner	John Underpass	18
5	Curry Leaf Corner	Rodriguez Mill	18
6	Spice Villa Grill	Reed Glens	18
7	Spice Villa Grill	Gary Green	18
8	Spice Villa Grill	Torres Mountain	18
9	Spice Villa Grill	Martinez Route	17
10	Curry Leaf Corner	Flores Vista	17

12. Which locations are prone to order cancellations?

Hint: Filter orders with `order_status = 'cancelled'`, join locations using `delivery_location_id`, group by `area_name`, count.

06:57 PM (2s)

```

SELECT
  l.area AS area_name,
  COUNT(o.order_id)
  AS cancellation_count
FROM alan_orders_bronze o
JOIN alan_locations_filtered l
ON o.delivery_location_id = l.location_id
WHERE LOWER(o.order_status) = 'cancelled'
GROUP BY l.area
ORDER BY cancellation_count DESC;

```

▶ (3) Spark Jobs

>  _sqldf: pyspark.sql.connect.dataframe.DataFrame = [a

Table  

	^A _C area_name	¹ ₂ ³ cancellation_count
1	Evans Stream	58
2	Elizabeth Locks	57
3	Flores Vista	55
4	Ashley Extensions	53
5	Hill Station	52
6	Padilla Mall	52
7	White Tunnel	51

13. Find top 10 customers who have ordered from the widest variety of restaurants.

Hint: Join orders, group by customer_id, count distinct restaurant_id, sort and pick top 10.

Just now (2s)

```
SELECT
  c.name AS customer_name,
  COUNT(DISTINCT o.restaurant_id) AS distinct_restaurants
FROM alan_orders_bronze o
JOIN alan_customers_filtered c
  ON o.customer_id = c.customer_id
GROUP BY
  c.customer_id,
  c.name
ORDER BY
  distinct_restaurants DESC,
  customer_name ASC
LIMIT 10;
```

► (4) Spark Jobs

>  _sqldf: pyspark.sql.connect.dataframe.DataFrame = [customer_name: str

Table  

	^A _C customer_name	¹ ₂ ³ distinct_restaurants	
1	Thomas Jennings	9	
2	Alejandro Rice	8	
3	Eric Carroll	8	
4	Mr. Eddie Wells MD	8	
5	Peter Conway	8	

14. Identify restaurants with frequent repeat customers.

Hint: Join orders, group by restaurant_id, customer_id, count, filter where count > 1.

Just now (2s)

```

SELECT
  r.name AS restaurant_name,
  c.name AS customer_name,
  COUNT(o.order_id) AS order_count
FROM alan_orders_bronze o
JOIN alan_restaurants_filtered r
  ON o.restaurant_id = r.restaurant_id
JOIN alan_customers_filtered c
  ON o.customer_id = c.customer_id
GROUP BY
  r.restaurant_id,
  r.name,
  c.customer_id,
  c.name
HAVING COUNT(o.order_id) > 1
ORDER BY
  order_count DESC,
  restaurant_name ASC,
  customer_name ASC;

```

► (4) Spark Jobs

> `_sqldf: pyspark.sql.connect.dataframe.DataFrame = [restaurant_name: string, customer_name: string, order_count: int]`

Table +

	^A _C restaurant_name	^A _C customer_name	¹ ₂ ₃ order_count
1	Curry Leaf Corner	Jose Alexander	4
2	Curry Leaf Corner	April Cummings	2
3	Curry Leaf Corner	Ashley Carey	2
4	Curry Leaf Corner	Ashley Thompson	2
5	Curry Leaf Corner	Brent Nichols	2

15. Which payment methods are most preferred for high-value orders across cuisines?

Hint: Filter orders with `total_amount > 40`, join restaurants, group by cuisine, payment_method, count.

06:57 PM (2s) 30

```

SELECT
  r.cuisines,
  o.payment_method,
  COUNT(o.order_id)
  AS usage_count
FROM alan_orders_bronze o
JOIN alan_restaurants_filtered r
ON o.restaurant_id = r.restaurant_id
WHERE o.total_amount > 40
GROUP BY r.cuisines, o.payment_method
ORDER BY r.cuisines, usage_count DESC;

```

▶ (3) Spark Jobs

> `_sqlcdf: pyspark.sql.connect.dataframe.DataFrame = [cuisines: string, payment_me`

Table +

	^A _C cuisines	^A _C payment_method	¹ ₃ usage_count
1	American	card	125
2	American	UPI	122
3	American	cash	118
4	American	wallet	117
5	American,Chinese	card	46

Problem Statements:

Guided Activity

Delta Live Tables

NOTE: To begin with, have orders details till June month in orders folder

`'dbfs:/FileStore/tables/roopa.bayya/dinedash/orders'`

Execute below in some notebook to move order details from June month onwards to other folder

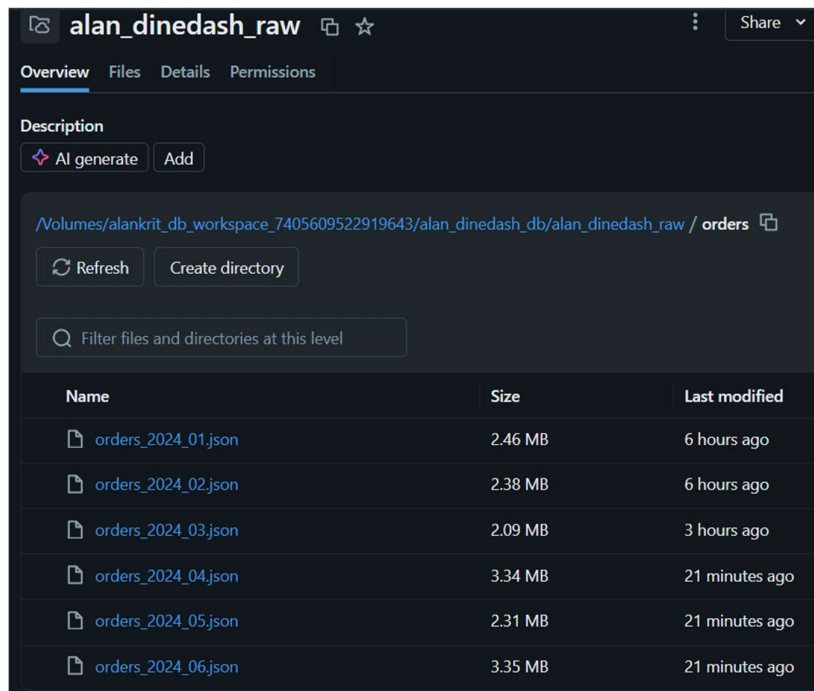
```

%python
for month in range(7, 13): # 07 to 12
    src =
    f"dbfs:/FileStore/tables/roopa.bayya/dinedash/orders/orders_2024_{month:02}.json"
    dst = f"dbfs:/FileStore/tables/roopa.bayya/dinedash/orders_2024_{month:02}.json"
    dbutils.fs.mv(src, dst)

```

Solution Phase 1:

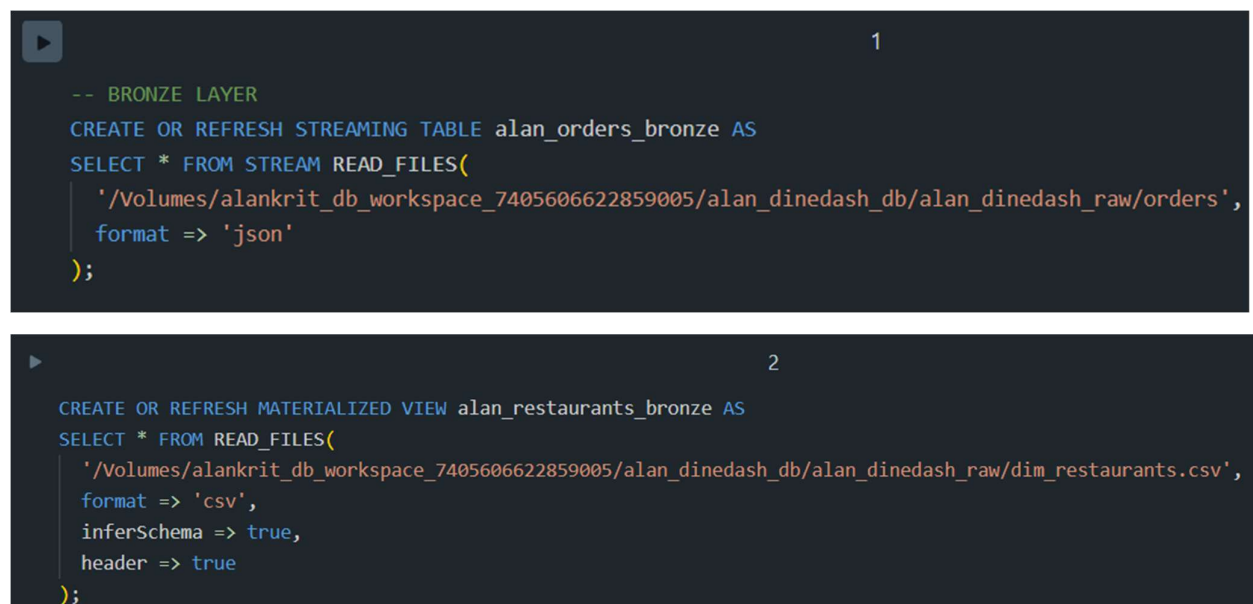
Here, we have used data till June month, we have used the Unity-Catalog Volume to replace legacy DBFS storage for our approach.



Authored a complete DLT SQL script defining the Bronze, Silver and Gold layers. Configured the DLT Pipeline in the Databricks UI to output to a dedicated schema (alan_dinedash_dlt_db) to securely isolate the automated tables from manual analytical environments.

Steps:

- a. For creation of pipeline, I have write the script as:



3

```

CREATE OR REFRESH MATERIALIZED VIEW alan_items_bronze AS
SELECT * FROM READ_FILES(
  '/Volumes/alankrit_db_workspace_7405606622859005/alan_dinedash_db/alan_dinedash_raw/dim_menu_items.csv',
  format => 'csv',
  inferSchema => true,
  header => true
);

```

4

```

CREATE OR REFRESH MATERIALIZED VIEW alan_customers_bronze AS
SELECT * FROM READ_FILES(
  '/Volumes/alankrit_db_workspace_7405606622859005/alan_dinedash_db/alan_dinedash_raw/dim_customers.csv',
  format => 'csv',
  inferSchema => true,
  header => true
);

```

5

```

CREATE OR REFRESH MATERIALIZED VIEW alan_agents_bronze AS
SELECT * FROM READ_FILES(
  '/Volumes/alankrit_db_workspace_7405606622859005/alan_dinedash_db/alan_dinedash_raw/dim_delivery_agents.csv',
  format => 'csv',
  inferSchema => true,
  header => true
);

```

6

```

CREATE OR REFRESH MATERIALIZED VIEW alan_locations_bronze AS
SELECT * FROM READ_FILES(
  '/Volumes/alankrit_db_workspace_7405606622859005/alan_dinedash_db/alan_dinedash_raw/dim_locations.csv',
  format => 'csv',
  inferSchema => true,
  header => true
);

```

7

```

-- SILVER LAYER
CREATE OR REFRESH STREAMING TABLE alan_cleaned_orders (
  CONSTRAINT valid_order_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_customer EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_items EXPECT (size(items_ordered) > 0) ON VIOLATION DROP ROW
)
AS
SELECT
  order_id,
  timestamp AS order_ts,
  customer_id,
  restaurant_id,
  agent_id,
  delivery_location_id,
  items_ordered,
  explode(items_ordered) AS item,
  total_amount,
  tip,
  payment_method,
  order_status
FROM STREAM(alan_orders_bronze);

```

```

CREATE OR REFRESH STREAMING TABLE alan_orders_silver (
  CONSTRAINT valid_restaurant_name EXPECT (restaurant_name IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_agent EXPECT (agent_name IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_item EXPECT (item_id IS NOT NULL) ON VIOLATION DROP ROW
)
AS
SELECT
  o.order_id,
  o.order_ts,
  o.customer_id,
  c.name AS customer_name,
  c.email,
  o.restaurant_id,
  r.name AS restaurant_name,
  r.cuisines,
  r.rating AS restaurant_rating,
  o.agent_id,
  a.name AS agent_name,
  a.rating AS agent_rating,
  o.delivery_location_id,
  l.city,
  o.item.item_id AS item_id,
  i.item_name,
  i.category,
  o.item.price AS item_price,
  o.item.quantity AS item_quantity,
  o.total_amount,
  o.tip,
  o.payment_method,
  o.order_status
FROM STREAM(alan_cleaned_orders) o
LEFT JOIN alan_customers_bronze c ON o.customer_id = c.customer_id
LEFT JOIN alan_restaurants_bronze r ON o.restaurant_id = r.restaurant_id
LEFT JOIN alan_agents_bronze a ON o.agent_id = a.agent_id
LEFT JOIN alan_items_bronze i ON o.item.item_id = i.item_id
LEFT JOIN alan_locations_bronze l ON o.delivery_location_id = l.location_id;

```

```

CREATE OR REFRESH STREAMING TABLE alan_orders_with_agents AS
SELECT
  o.order_id,
  o.order_ts,
  o.agent_id,
  a.name AS agent_name,
  a.rating AS agent_rating,
  o.tip
FROM STREAM(alan_cleaned_orders) o
LEFT JOIN alan_agents_bronze a ON o.agent_id = a.agent_id;

```

10

```

CREATE OR REFRESH STREAMING TABLE alan_orders_with_items AS
SELECT
  o.order_id,
  o.order_ts,
  o.restaurant_id,
  exploded.item_id,
  i.item_name,
  i.category,
  exploded.price,
  exploded.quantity
FROM (
  SELECT order_id, timestamp AS order_ts, restaurant_id, explode(items_ordered) AS exploded
  FROM STREAM(alan_orders_bronze)
) o
LEFT JOIN alan_items_bronze i ON o.exploded.item_id = i.item_id;

```

11

```

CREATE OR REFRESH STREAMING TABLE alan_orders_with_locations AS
SELECT
  o.order_id,
  o.order_ts,
  o.delivery_location_id,
  l.city,
  l.state,
  l.area
FROM STREAM(alan_cleaned_orders) o
LEFT JOIN alan_locations_bronze l ON o.delivery_location_id = l.location_id;

```

12

```

CREATE OR REFRESH STREAMING TABLE alan_orders_with_customers AS
SELECT
  o.order_id,
  o.customer_id,
  c.name AS customer_name,
  c.location_id AS customer_location_id,
  o.total_amount,
  o.tip,
  o.payment_method,
  o.order_status
FROM STREAM(alan_cleaned_orders) o
LEFT JOIN alan_customers_bronze c ON o.customer_id = c.customer_id;

```

13

```
-- GOLD LAYER
CREATE OR REFRESH MATERIALIZED VIEW alan_total_orders_per_restaurant AS
SELECT restaurant_name, COUNT(DISTINCT order_id) AS total_orders
FROM alan_orders_silver
GROUP BY restaurant_name;
```

14

```
CREATE OR REFRESH MATERIALIZED VIEW alan_top_selling_items AS
SELECT item_name, SUM(item_quantity) AS total_qty_sold
FROM alan_orders_silver
GROUP BY item_name
ORDER BY total_qty_sold DESC
LIMIT 10;
```

15

```
CREATE OR REFRESH MATERIALIZED VIEW alan_avg_fees_by_city AS
SELECT city, AVG(restaurant_rating) AS avg_restaurant_rating, AVG(tip) AS avg_tip
FROM alan_orders_silver
GROUP BY city;
```

16

```
CREATE OR REFRESH MATERIALIZED VIEW alan_agent_order_counts AS
SELECT agent_name, COUNT(DISTINCT order_id) AS orders_handled, AVG(agent_rating) AS avg_rating
FROM alan_orders_silver
GROUP BY agent_name
ORDER BY orders_handled DESC;
```

17

```
CREATE OR REFRESH MATERIALIZED VIEW alan_daily_revenue_trend AS
SELECT DATE(order_ts) AS order_date, SUM(total_amount) AS total_revenue
FROM alan_orders_silver
GROUP BY order_date
ORDER BY order_date;
```

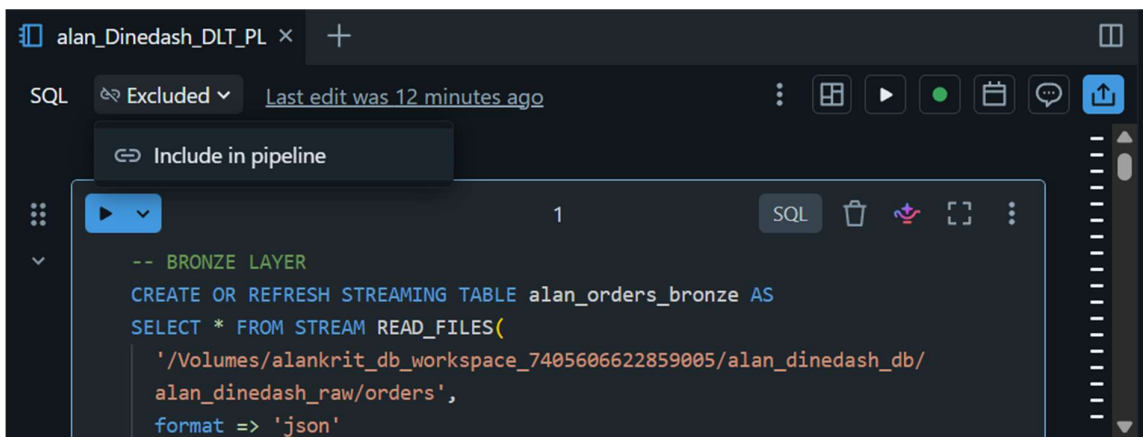
18

```
CREATE OR REFRESH MATERIALIZED VIEW alan_customer_ltv AS
SELECT customer_id, customer_name, SUM(total_amount) AS lifetime_spend
FROM alan_orders_silver
GROUP BY customer_id, customer_name
ORDER BY lifetime_spend DESC;
```

19

```
CREATE OR REFRESH MATERIALIZED VIEW alan_customer_order_counts AS
SELECT customer_name, COUNT(*) AS total_orders
FROM alan_orders_with_customers
GROUP BY customer_name;
```

- b. Include the alan_dinedash_dlt_pl notebook in the pipeline and name the pipeline as alan_dinedash_etl_pipeline



- c. Created a new schema named: alan_dinedash_dlt_db for the pipeline execution

Create a new schema

A schema is the second layer of Unity Catalog's three-level namespace and is used to organize tables, views, volumes and other objects. [Learn more](#)

Schema name*

alan_dinedash_dlt_db

Storage location

Cloud storage location used for managed tables and volumes in this schema. If not specified, it defaults to the metastore root location.

Select external location sub/path

[Create a new external location](#)

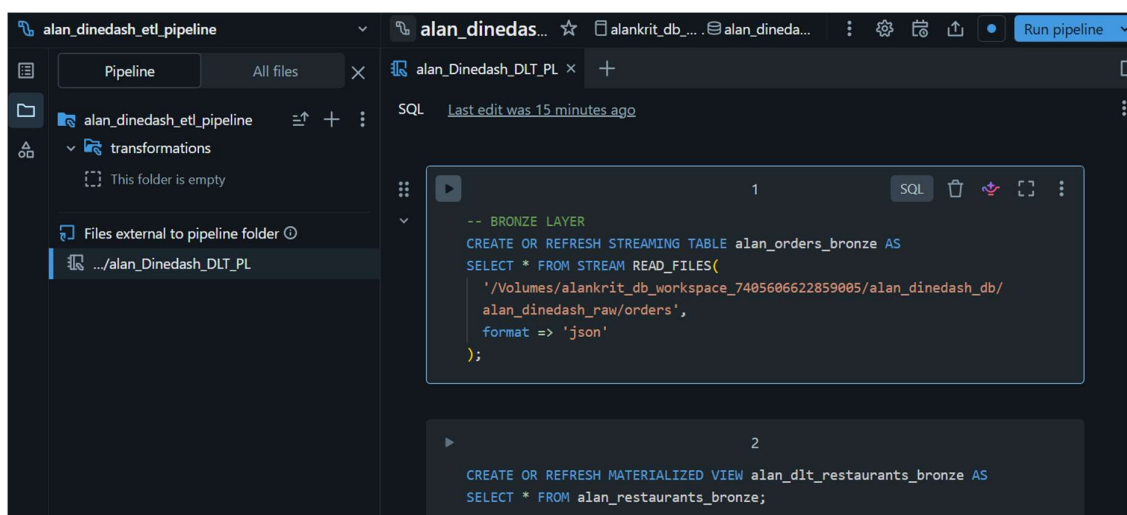
Access requests

Requests will be sent to fy26databricks5thaugto7thsepuser5@acp11792b.onmicrosoft.com. This can be configured after creation.

Comment

Cancel Create

d. Click on Run Pipeline at the top right as:

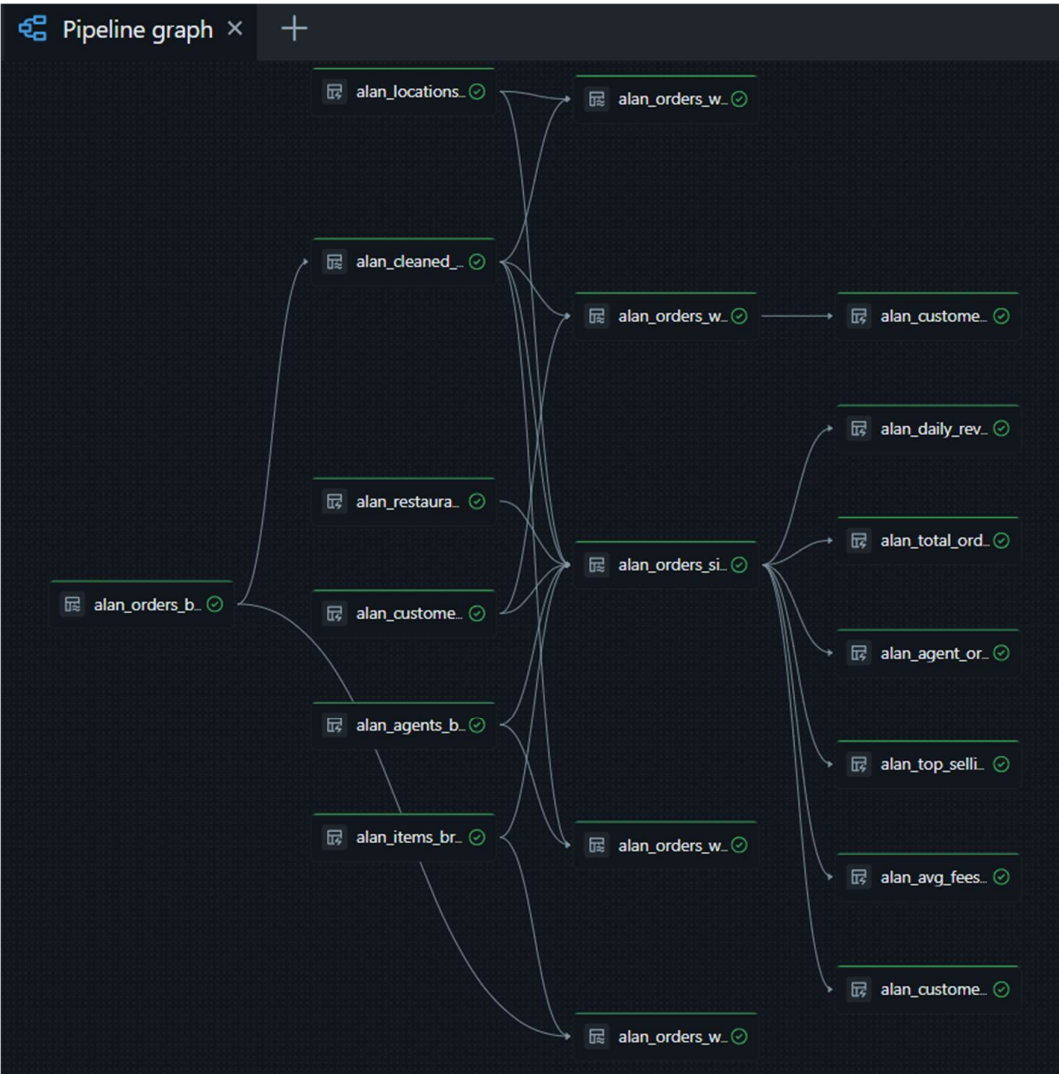


e. After click on Run Pipeline and waiting for 2-3 minutes, we get like this:

The screenshot shows the 'Tables' tab in the Databricks interface for the 'alan_dinedash_etl_pipeline'. The table lists various data tables with their duration, output size, and status. The 'Incremental...' column shows the status of each table, such as 'Full recompute' or 'N/A'.

Name	Duration	Output...	Expectations	Incrementali...
alan_agent_order_counts	6s	400	-	Full recompute
alan_agents_bronze	4s	450	-	Full recompute
alan_avg_fees_by_city	6s	101	-	Full recompute
alan_cleaned_orders	4s	63K	3 met	N/A
alan_customer_itv	6s	9.4K	-	Full recompute
alan_customer_order_counts	6s	9.2K	-	Full recompute
alan_customers_bronze	4s	10K	-	Full recompute
alan_daily_revenue_trend	4s	168	-	Full recompute
alan_items_bronze	4s	500	-	Full recompute
alan_locations_bronze	5s	100	-	Full recompute
alan_orders_bronze	9s	38K	-	N/A
alan_orders_silver	4s	59K	2 met 1 unmet	N/A
alan_orders_with_agents	3s	77K	-	N/A
alan_orders_with_customers	3s	63K	-	N/A

f. We also get the pipeline graph of connection of tables with each other as:



g. We can observe the specific details for the table; e,g; Here observe the details of alan_orders_bronze table:

Filter by flow name

alankrit_db_workspace_740560662285900
5.alan_dinedash_dlt_db.alan_orders_bronze

Flow ID

eaa645cc-7b6c-4ca9-8228-ad26704e7452

Flow type

Append

Flow status

Completed

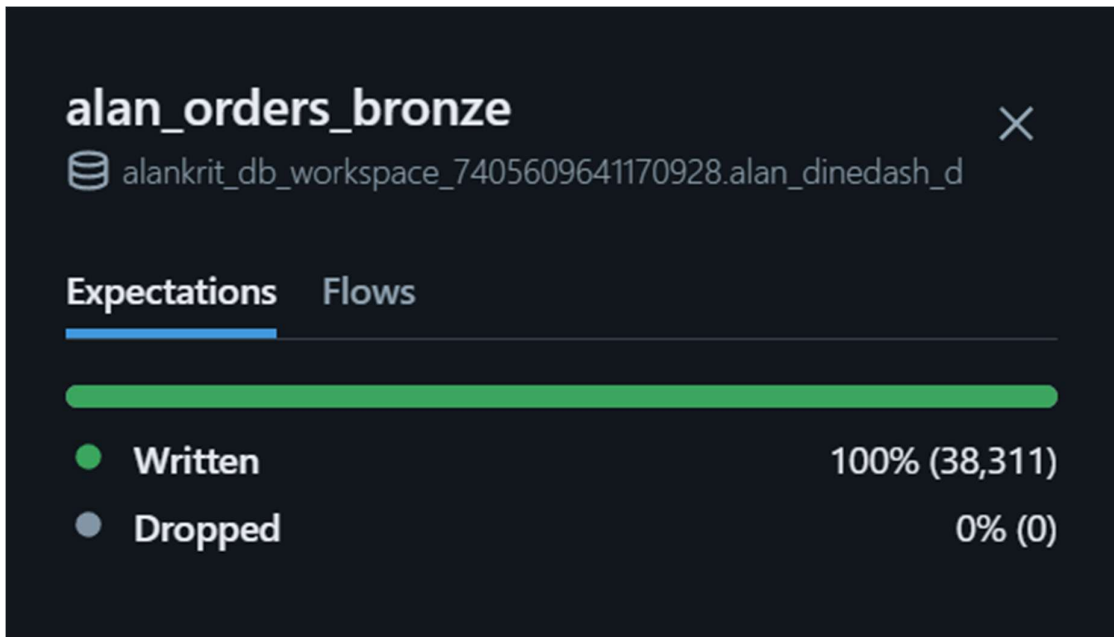
Start time

Aug 21, 2026, 04:00 PM

Duration

7s

Reset and refresh



Streaming tab... 9s

alan_orders_bronze ✓

Output records: 38K

← All tables **alan_orders_bronze** > Data **Columns** Table metrics Perform

⤴ Expand all ⤵ Collapse all

agent_id: string
customer_id: string
delivery_location_id: string
> **items_ordered:** array
order_id: string
order_status: string
payment_method: string
restaurant_id: string
timestamp: string
tip: double
total_amount: double
_rescued_data: string

Here, we can observe the schema, output records, expectations-written and dropped records etc.

Solution Phase 2:

Here, we have added data of July and August Month also

alan_dinedash_raw

OverviewFilesDetailsPermissions

Description

AI generateAdd

/Volumes/alankrit_db_workspace_7405609522919643/alan_dinedash_db/alan_dinedash_raw / orders

RefreshCreate directory

Filter files and directories at this level

Name	Size	Last modified
orders_2024_01.json	2.46 MB	7 hours ago
orders_2024_02.json	2.38 MB	7 hours ago
orders_2024_03.json	2.09 MB	4 hours ago
orders_2024_04.json	3.34 MB	1 hour ago
orders_2024_05.json	2.31 MB	1 hour ago
orders_2024_06.json	3.35 MB	1 hour ago
orders_2024_07.json	2.70 MB	37 seconds ago
orders_2024_08.json	3.19 MB	37 seconds ago

h. Run the pipeline again and observe the changes

alan_dinedash_etl_pipeline

Pipeline graph

Streaming4s

alan_orders_bronze

Output records: 52K

orders_bronze > DataColumnsTable metricsPerformance

DurationOutput re...Expectations

4s52K-

alan_orders_bronze

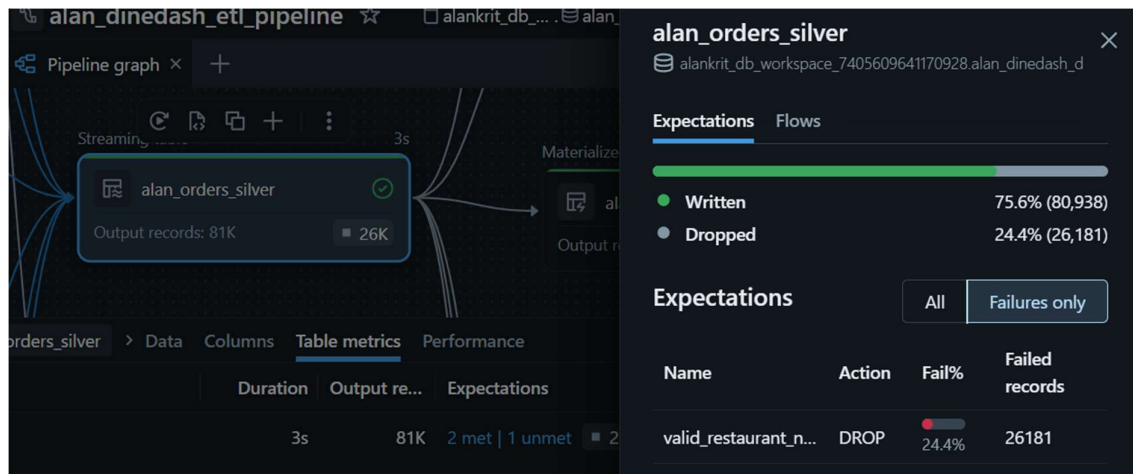
alankrit_db_workspace_7405609641170928.alan_dinedash_d

ExpectationsFlows

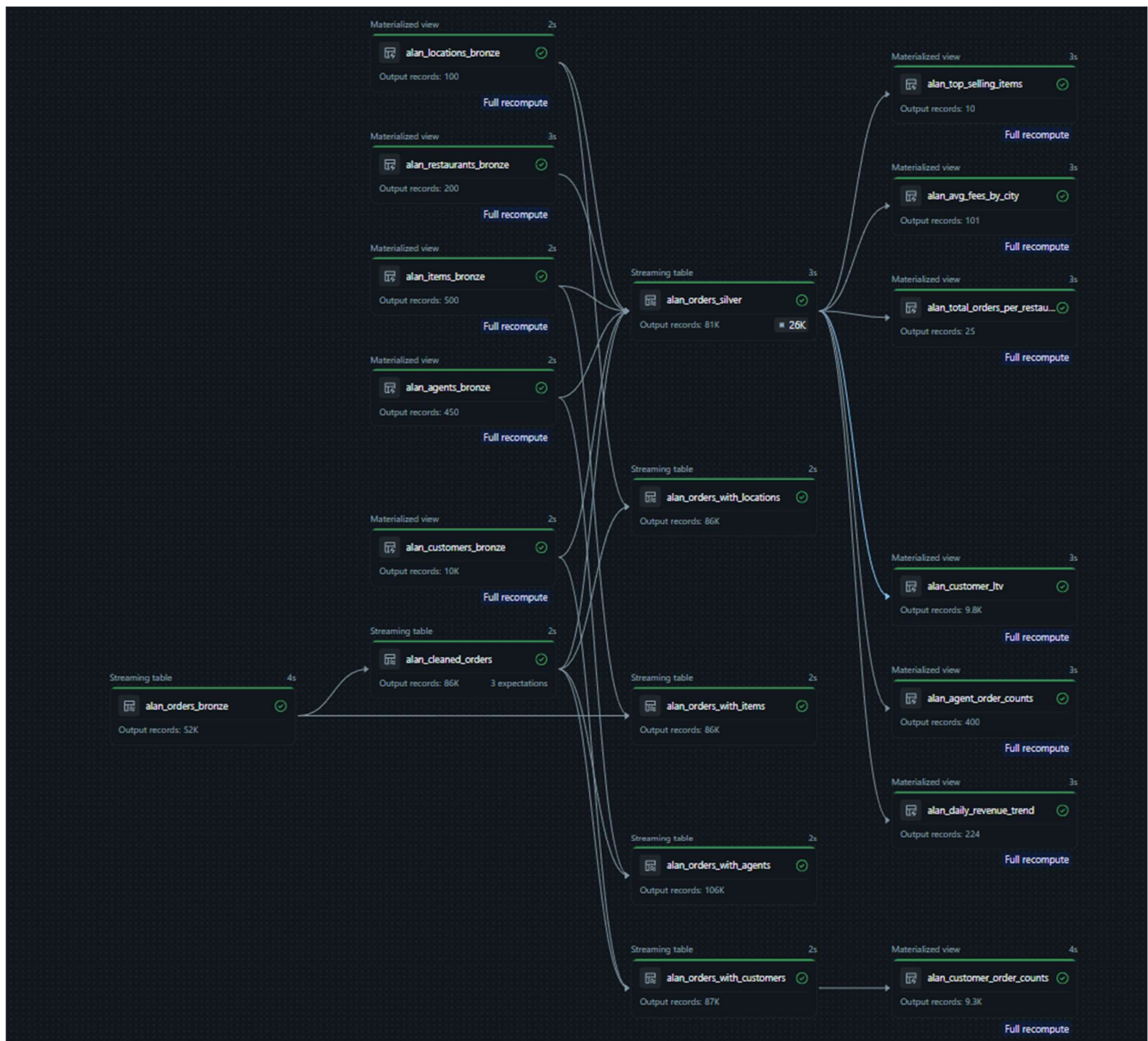
Written100% (52,467)

Dropped0% (0)

i. Observe the expectations in alan_orders_silver table, written and dropped values of the new added values



j. This is the final table graph till now we observe as:



Problem Statements:

Workflows

You will create a workflow to run the pipeline and publish some important insights onto Dashboard.

For this you create a Job in Databricks Workspace.

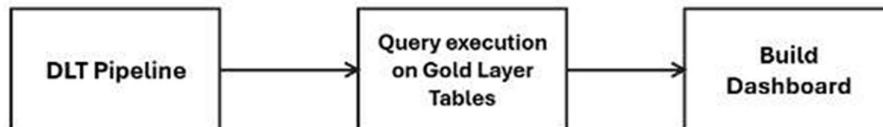
The job you create will have following flow:

Task 1 : DLT Pipeline (which you have created earlier) execution

Task 2 : Creating Dashboard to display total_orders_per-restaurant, daily_revenue_trends, top_selling_items, total_revenue

Dashboards are fed from gold layer table query results and we add visualization on top of it

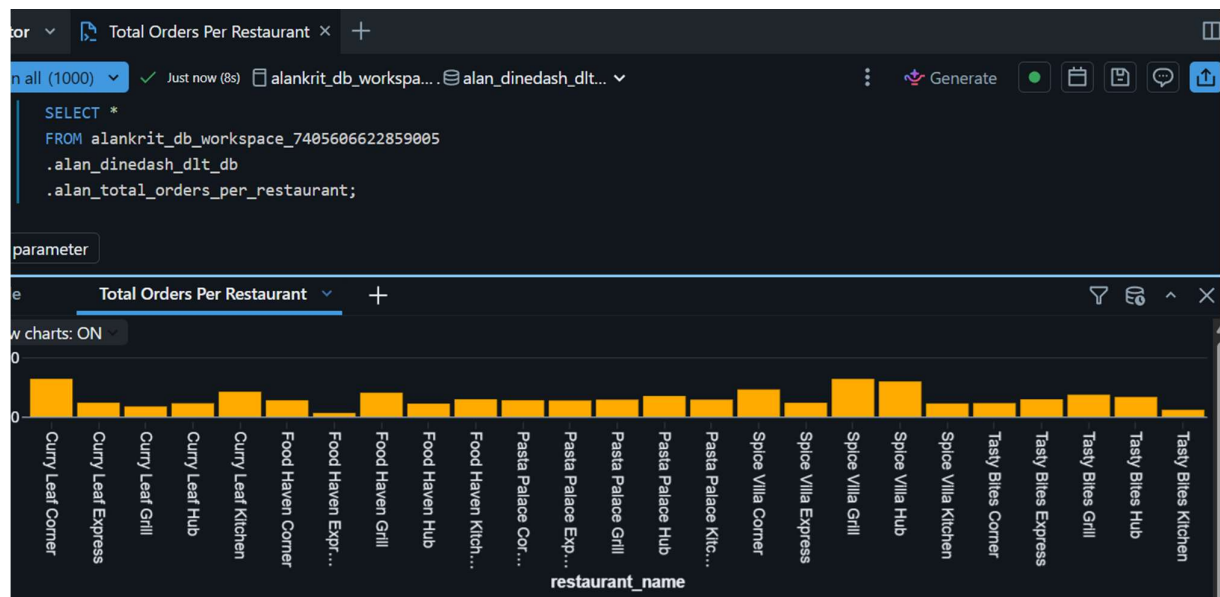
So, first create the queries as shown below and save them



Solution:

Authored and saved four dedicated SQL queries against the alan_dinedash_dlt_db schema within the SQL Editor. Generated visualizations (Bar, Pie, Line, and Counter) from the raw results and published them to a unified Legacy Dashboard named "Dine Dash Metrics Dashboard".

- SQL Query for Total Orders per Restaurant with Visualization, then Create and adding it to DineDash Metrics Dashboard



Add to dashboard ✕

☒ Create new dashboard
 ☐ Add to existing dashboard

Name

DineDash Metrics Dashboard

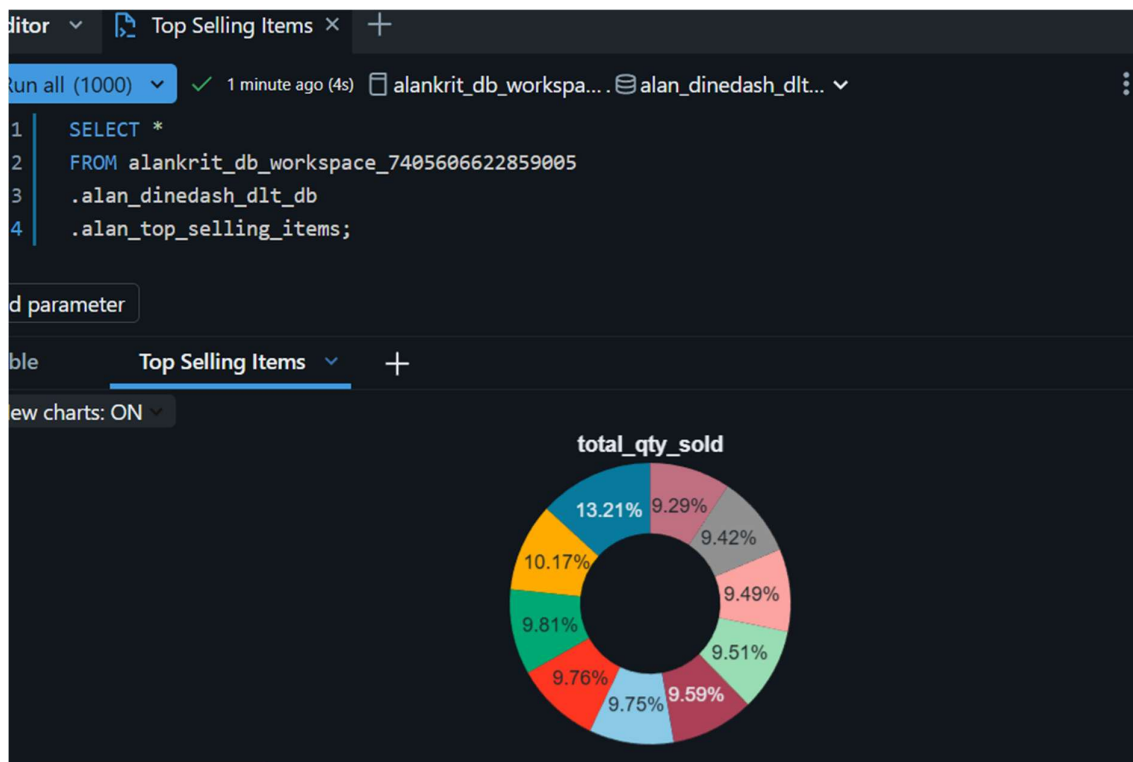
Visualizations (1 of 1 selected)

☒ Name

☒  Total Orders Per Restaurant

Cancel Create

- SQL Query for Top Selling Items with Visualization and adding it to existing DineDash Metrics Dashboard



Add to dashboard

☐ Create new dashboard ☒ Add to existing dashboard

Name

DineDash Metrics Dashboard

Visualizations (1 of 1 selected)

☒ Name

☒  Top Selling Items

Cancel

Import

- SQL Query for Daily Revenue Trend with Visualization and adding it to existing DineDash Metrics Dashboard



Add to dashboard ✕

☐ Create new dashboard ☒ Add to existing dashboard

Name

DineDash Metrics Dashboard ✕ ▼

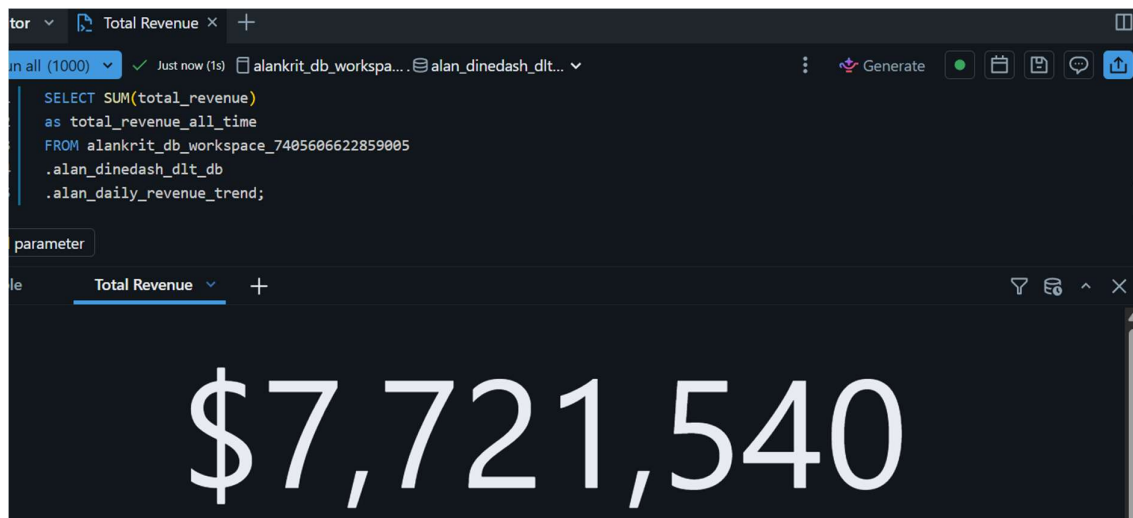
Visualizations (1 of 1 selected)

☒ Name

☒  Daily Revenue Trend

Cancel Import

- SQL Query for Total Revenue with Visualization and adding it to existing DineDash Metrics Dashboard



Add to dashboard

☐ Create new dashboard
 ☒ Add to existing dashboard

Name

DineDash Metrics Dashboard

Visualizations (1 of 1 selected)

☒ Name

☒

10k

 Total Revenue

Cancel

Import

Here, we have implemented 4 SQL Queries and generated the Visualizations for them and added them to Dashboard

- After this, we have added all the visualizations to the DineDash Metrics Dashboard and publish it as:



Screenshot: The fully populated "Dine Dash Metrics Dashboard" showing the four visualizations and the total revenue counter.

Problem Statements:

Creating Workflow/Job

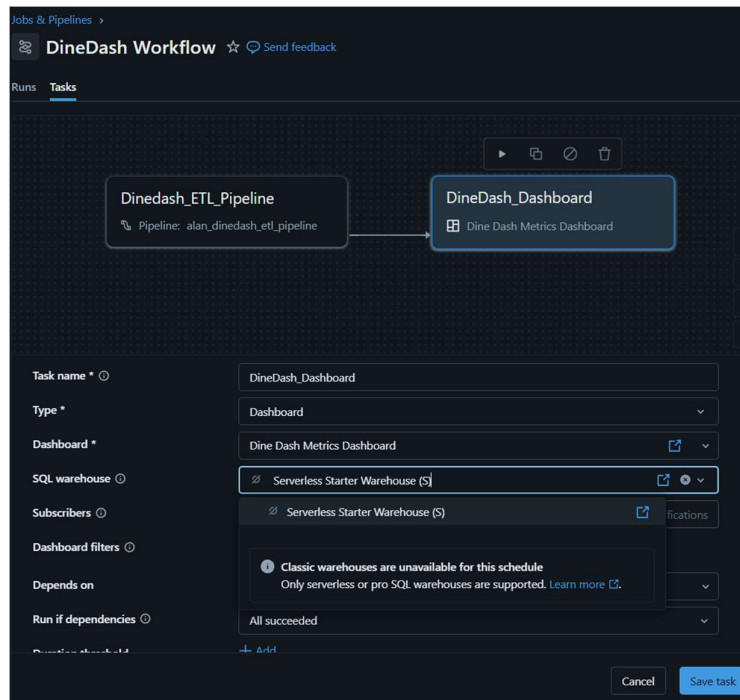
Solution: Configured a Databricks Workflow (Job) with two sequential tasks. Task 1 was configured to execute the Dinedash_ETL_Pipeline (DLT). Task 2 was configured to refresh the Dine Dash Metrics Dashboard, with a strict dependency requiring Task 1 to succeed before Task 2 could initiate.

Steps:

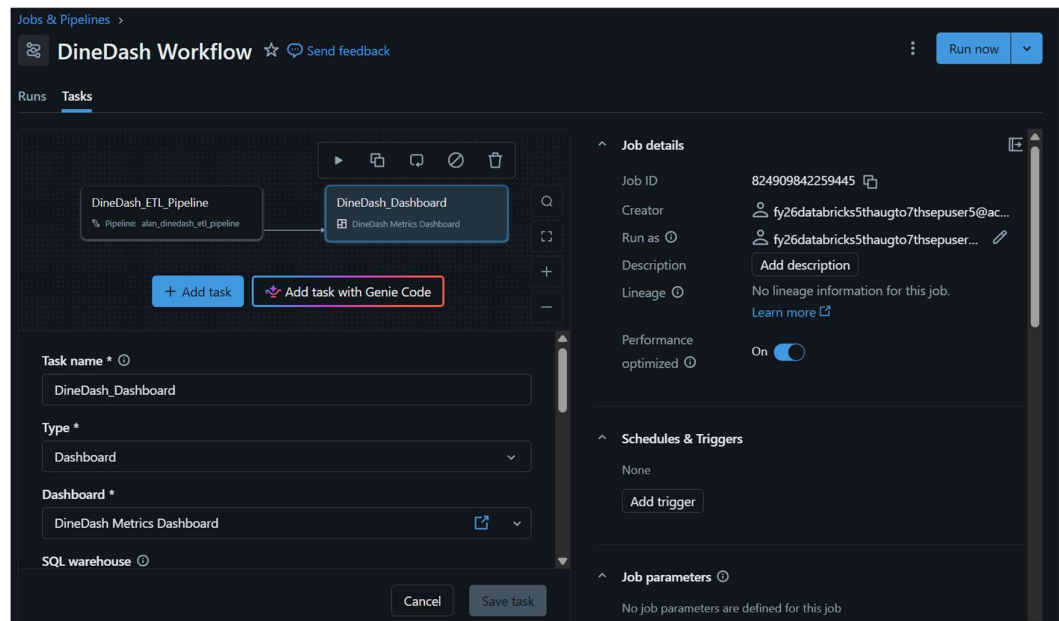
- a. Create a task by clicking on Add Task i.e.; DineDash_ETL_Pipeline and connect your earlier created pipeline: alan_dinedash_etl_pipeline

The screenshot shows the 'Jobs & Pipelines' section in Databricks. The 'DineDash Workflow' is selected, and the 'Tasks' tab is active. A task named 'Dinedash_ETL_Pipeline' is being configured, linked to the 'alan_dinedash_etl_pipeline'. Below the task name, the 'Type' is set to 'Pipeline', and the 'Pipeline' dropdown is set to 'alan_dinedash_etl_pipeline'. A checkbox 'Trigger a full refresh on the pipeline' is checked. Buttons for '+ Add task' and '+ Add task with Genie Code' are visible.

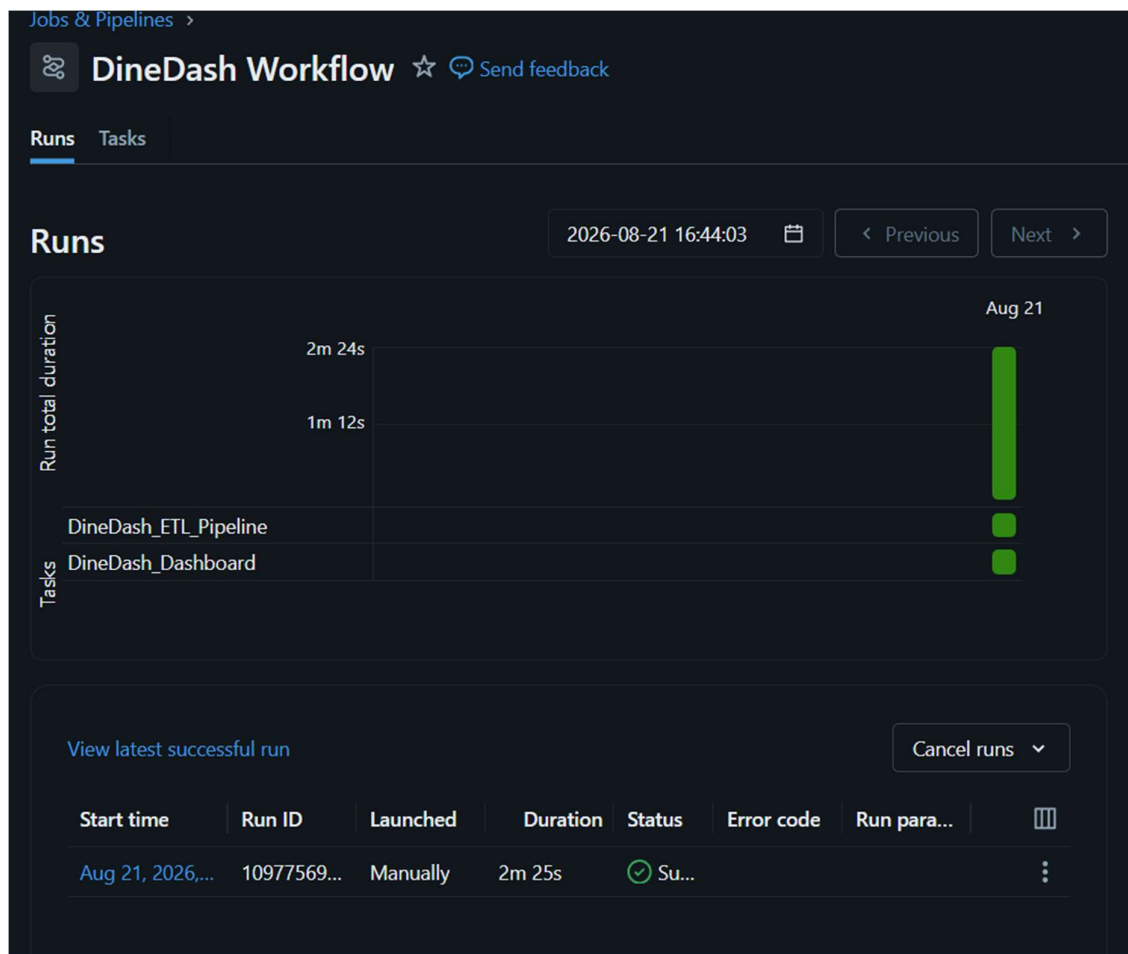
- b. Create a new task and add your Dashboard created earlier i.e; DineDash Metrics Dashboard and make it dependent on Pipeline Task



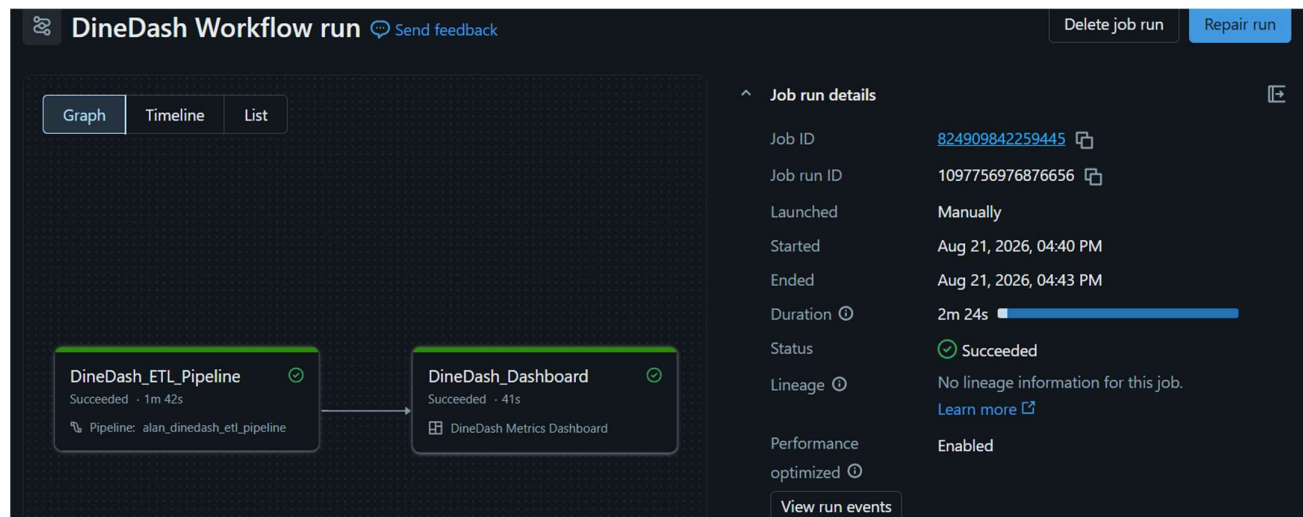
c. Click on Run Now on top right and wait for 3-4 minutes to complete it



d. Observe the Runs tab after successful execution of Workflow as:

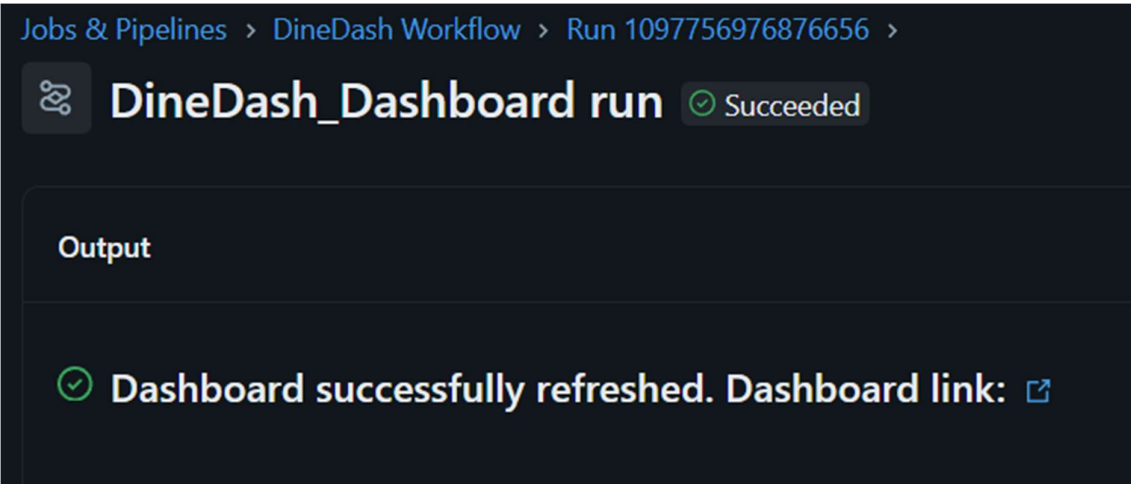
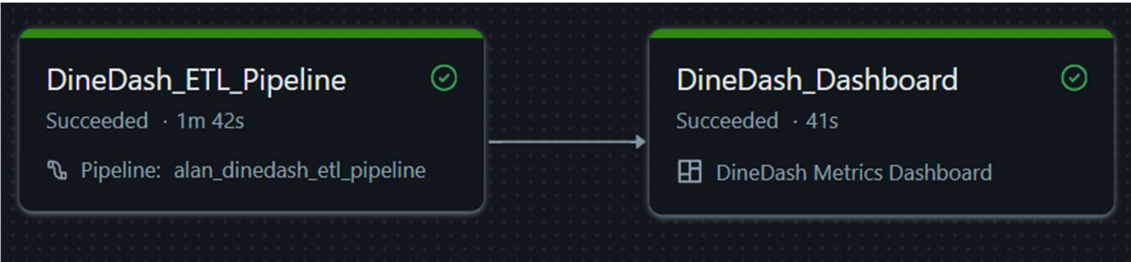


- e. Click on workflow and observe the green graphs indicates the successful execution of workflow



Screenshot: The Databricks Workflows "Job details" showing both the Pipeline and Dashboard tasks marked as "Succeeded" in green.

f. Click on DineDash_Dashboard tab here and click on hyperlink observe the dashboard



g. You can see the DineDash_Dashboard from here



h. Upload the September to December orders month datasets in the Volume as:

